jamk

Impact of Generative Artificial Intelligence in Software Development Life Cycle

A Mixed-Methods Study in Finland and Abroad Using Concurrent Triangulation Design Approach

Fahad Mohamed

Master's thesis
April 2025
Master's Degree Programme in Information Technology,
Full-Stack Software Development

jamk | Jyväskylän ammattikorkeakoulu
University of Applied Sciences

jamk

Description

Mohamed Fahad Hamad Rashid

Impact of Generative AI in Software Development Life Cycle, A Mixed Methods Study in Finland and Abroad and Concurrent Triangulation Design Approach

Jyväskylä: Jamk University of Applied Sciences, April 2025, 69 pages

Master’s Degree Programme in information Technology. Master’s thesis.

Permission for open access publication: Yes

Language of publication: English

Abstract

Generative Artificial Intelligence (AI) is revolutionising the Software Development Life Cycle (SDLC) by enhancing productivity, efficiency, and collaboration, especially in coding, testing, and documentation. This study examines the adoption and impact of generative AI tools in SDLC phases within Finland and internationally, aiming to address the growing integration of AI in software development.

The research was conducted to assess how AI influences SDLC phases, exploring both its advantages and challenges. A mixed-methods approach with a concurrent triangulation design to validate and cross-verify findings was employed. It combined surveys and secondary data to provide a comprehensive understanding. The quantitative survey analysed adoption rates and user perceptions, while secondary data validated the findings with broader industry perspectives.

Findings reveal that AI significantly improves coding and testing efficiency, but adoption remains limited in earlier phases like requirement gathering and design due to complexity and subjectivity. Developers widely adopt AI for routine tasks, reporting increased productivity, though they express concerns about job displacement, data security, and ethical risks.

These results highlight the importance of improving AI's reliability and contextual understanding to maximise its potential. Addressing these challenges could pave the way for AI's broader adoption in software development. Future research should focus on advancing AI capabilities in early SDLC phases, such as requirement gathering, to enhance its value in complex tasks.

Keywords/tags (subjects)

AI (Artificial Intelligence), software development, life cycle, survey, Generative artificial intelligence, Concurrent Triangulation

Miscellaneous (Confidential information)

Appendices include supporting materials such as survey forms, data tables, and charts.
Participants' email addresses will remain confidential and used solely for this study.

Contents

Abbreviations 4
1 Introduction 5
1.1 Background of the Study 5
1.2 Problem Statement 6
1.3 Purpose and Objectives 7
1.4 Research questions 7
1.5 Relevance to Sustainable Development Goals 8
1.6 Scope and Limitations 8
1.7 Thesis Structure 9
2 Literature Review 10
2.1 History of Software Development Life Cycle 10
2.2 History of Artificial Intelligence. 13
2.3 Software Development Life Cycle 14
2.4 Generative Artificial Intelligence in Software development Phases 15
2.4.1 Artificial Intelligence in Requirements Identification and Gathering 16
2.4.2 Artificial Intelligence in Design and Architecture 17
2.4.3 Artificial Intelligence in Development and Programming 17
2.4.4 Artificial Intelligence in Testing 18
2.4.5 Artificial Intelligence in Deployment and Maintenance 18
2.5 Impact of Generative AI in Software Development Process 19
2.6 Theoretical Framework 25
3 Research Implementation 28
3.1 Research Method 28
3.2 Data Collection 28
3.3 Data Analysis 30
4 Results 32
4.1 Background and Demographic 32
4.2 Adoption of GenAI in Software Development Life Cycle Phases. 34
4.3 Positive and Negative Impacts of Generative AI on SDLC 36
4.4 Perceptions and Experiences of Developers Regarding Generative AI 39
4.5 Future of Generative AI in SDLC 40
5 Discussion 43
5.1 Ethical Foundation and Reliability 43

5.2 Discussion of the main results in view of Concurrent Triangulation Design 44
5.2.1 Adoption rate of Generative AI in Software Development Life Cycle. 44
5.2.2 Positive and Negative Impacts 45
5.2.3 Perceptions and Experiences of Developers Regarding Generative AI 46
5.2.4 Future of Artificial Intelligence in SDLC 47
5.3 Reconciling Theoretical Frameworks and Data collected. 48
5.4 Recommendation for Future Research 49
6 Conclusion 50
References 52
Appendices 57
Appendix 1 The Impact of Generative AI on Software Development Life Cycle survey 57
Appendix 2 Survey results 62

Figures

Figure 1. Representation of different phases of the waterfall model (adapted from Sommerville, 2016, p.47) 11
Figure 2. Agile model approach with sprints (adapted from SolDevelo, 2020) 12
Figure 3. Adoption of Artificial intelligence (McKinsey & Company, 2024). 21
Figure 4. AI tool's ability to handle complex tasks (Stack Overflow, 2024). 22
Figure 5. Benefits of AI tools (Stack Overflow, 2024). 23
Figure 6. Participants in the survey by regions 32
Figure 7. Experience of participants based on years 33
Figure 8. Usage of Generative AI in organisation 33
Figure 9. Current role of participant in software development 34
Figure 10. Software models used in organisations or teams 34
Figure 11. Phases of the SDLC that implement AI 62
Figure 12. Impact of GenAI in Requirement and Gathering 62
Figure 13. Impact in design and architecture Phase 63
Figure 14. Impact in coding and implementation phase 63
Figure 15. Impact in testing and QA Phase 64
Figure 16. Deployment and maintenance Phase 64
Figure 17. Efficiency in development workflow results 64
Figure 18. Productivity improvement results 65
Figure 19. Decision making results 65
Figure 20. Impact of AI In innovation results 66

Figure 21. Challenges in integrating Gen AI into SDLC results 66
Figure 22. Ethical concerns results 66
Figure 23. Overall experience using generative AI results 67
Figure 24. Perceptions of respondents about the future of AI in SDLC 67
Figure 25. Comments and experiences shared regarding generative AI in software development by the respondents 67

Tables

Table 1. Calculated Weighted Average adoption scores in Software Development Life cycle phases from primary data. 35
Table 2. Secondary data collected from different research indicating where GenAI has being implemented in SDLC 36
Table 3. Survey showing the positive and negative impacts in SDLC from primary data 37
Table 4. Positive impacts data from secondary sources 38
Table 5. Negative impact data obtain from secondary source 38
Table 6. Developer Perceptions and Experiences with Generative AI from primary data 39
Table 7. Perception and experience of developers from secondary data 40
Table 8. Future themes from Primary data 41
Table 9. Future trends from secondary data 42

Abbreviations

AI	Artificial Intelligence
ALM	Application Life Management
API	Application Programming Interface
CD	Continuous Deployment
DOI	Diffusion of Innovation (Theory)
GAN	Generative Adversarial Network
GenAI	Generative Artificial Intelligence
GPT	Generative Pre-trained Transformer
IDE	Integrated Development Environment
IJCRT	International Journal of Creative Research
LLM	Large Language Model
Llama	Large Language Model Meta AI
MCP	McCulloch-Pitts Neuron
MITRE	Corporation (nonprofit tech organization)
NIST	National Institute of Standards and Technology
NLP	Natural Language Processing
PaLM	Pathway Language Models
RAD	Rapid Application Development
SADT	Structured Analysis and Design Technique
SDLC	Software Development Life Cycle
SREM	Software Requirements Engineering Methodology
TPC	Technology to Performance Chain
TTF	Task-Technology Fit (Model)
UI	User Interface
UX	User Experience
V&V	Verification and Validation
XP	Extreme Programming

1 Introduction

1.1 Background of the Study

The Software Development Life Cycle is a structured approach used by software development teams to create high-quality software in a cost-effective and time-efficient manner. Its main goal is to reduce project risks through careful planning, ensuring that the final product meets customer expectations both during and after production. Software Development Life Cycle (SDLC) breaks down the software development process into specific stages, making It easier to assign task, track progress, and measure outcomes effectively (Amazon web Service, n.d).

De Vicente Mohino, Bermejo Higuera, Bermejo Higuera, and Sicilia Montalvo (2019) describe the software life cycle as a framework outlining the process organizations use to develop an application from inception to the end of its lifecycle. Various software development methodologies exist, all offering a high-level perspective. Despite their differences, the core activities in each methodology include the following: Requirement identification and gathering, Design and architecture, Development and Coding, Testing, Deployment and Maintenance of the application.

While the Software Development Life Cycle focuses on structured approaches to software creation, it forms part of a broader framework known as Application Lifecycle Management (ALM). ALM spans the entire lifecycle of an application, encompassing governance, development, and operations, ensuring continuous value delivery (Chappell, 2008).

The Software Development Life Cycle consist of multiple phases, each critical for delivering high quality software that meets user requirements and client goals. Traditionally these phases rely heavily on manual efforts, human expertise, and repetitive process. However, the advent of artificial intelligence has introduced a significant shift in how these phases are executed. The manual nature of the traditional SDLC often leads to inefficiencies, errors, and delays. Integrating Artificial intelligence (AI) into the SDLC presents an opportunity to overcome these challenges and enhance software development practices (Smith & Wei,2024).

Artificial Intelligence is a powerful technology that helps machines act like humans by learning, understanding language, solving problems, and making decisions on their own. AI can recognize objects, understand speech or text, and improve over time by learning from experience. It can analyse information, give useful advice, and handle tasks that save time and effort. This makes AI a key part of modern technology, improving lives and industries worldwide (Stryker & kavlakogula.2024).

According to Black, Davenport, Olszewska, Rößler, Smith, & Wright, (2022), artificial intelligence has been around since the mid-20th century and has become the "new electricity", making it a must have for many businesses. The impact AI is increasingly becoming evident in various sectors. The large language models are demonstrating a human-like competence in so many areas even as far as image recognition and natural language understanding (Russo,2024).

While the SDLC has undergone significant evolution since its early days marked by the introduction of the waterfall model in 1970 by Winston W. Royce (Parkash, 2024), the integration of AI represents a pivotal shift in software development practices. By blending traditional frameworks with cutting-edge AI technologies, organizations may achieve greater efficiency and productivity, ultimately transforming the software engineering landscape.

This study seeks to explore how Generative AI impacts the Software Development Life Cycle. It adopts a mixed-methods approach, gathering data both within Finland and internationally. The author collects information through surveys featuring both open-ended and close-ended questions, while also integrating recently done secondary research to validate and complement the survey results. By leveraging the strengths of secondary data, the research addresses potential limitations in the survey findings. The outcomes of this study are intended to provide valuable insights for students in software development courses as well as for a wider audience, including software developers, managers, and other interested stakeholders.

1.2 Problem Statement

In recent years, the integration of AI into the software development lifecycle has become increasing prevalent. This technology shift has significantly transformed traditional development

process, introducing new methodology and tools that are reshaping the way software is designed, developed, tested and maintained.

Major technology companies such as google have pioneered the use of generative AI to optimize operations, including improving efficiency of data centres, Additionally, AI powered tools like intelliCode in Microsoft Visual studio code exemplify how machine learning is being embedded into the integrated development environment to enhance developer productivity by predicting coding patterns and suggesting intelligent suggestions (Svyatkovskiy, Deng, Fu, & Sundaresan, 2020).

AI tools continue to be adopted across various stages of the software development life cycle (SDLC), there is a growing need to examine their impact of the software development life cycle in each phase. Despite the rapid integration of AI, there remains a lack of comprehensive research into how AI models have impacted the software development life cycle. This study aims to explore the impact of generative AI in the software development life cycle its negative and positive aspects. It also explores the perception that developers have towards the introduction of generative AI in the software development life cycle.

1.3 Purpose and Objectives

The purpose of this thesis is to examine the impact of generative artificial intelligence (GenAI) on the SDLC in Finland and abroad. The study aims to analyse the effects of GenAI on different SDLC phases, adoption patterns, and its future role in software development. Additionally, it seeks to gather the perceptions and experiences of software developers regarding GenAI, while validating findings with prior but recent research. The results of this study aim to provide insights beneficial to software development students, developers, managers, and other stakeholders.

1.4 Research questions

1. How does Generative AI impact different phases of the Software Development Life Cycle?
2. What are the adoption trends of Generative AI in the Software Development Life Cycle?
3. What is the future of Generative AI in the Software Development Life Cycle?

4. What are the perceptions and experiences of software developers regarding Generative AI?
5. What are the benefits and challenges of Generative AI in the Software Development Life Cycle?

1.5 Relevance to Sustainable Development Goals

This research aligns with several United Nations Sustainable Development Goals (SDGs), reinforcing its societal and environmental relevance. First, the study contributes to Decent Work and Economic Growth (SDG 8) by examining how GenAI automates repetitive development tasks, allowing developers to focus on higher-value work. This supports productivity and the evolution of new AI-related roles. By accelerating innovation in digital systems, GenAI's role in software development aligns with Industry, Innovation and Infrastructure (SDG 9), helping modernize workflows across sectors. The research also supports Responsible Consumption and Production (SDG 12) by showcasing how GenAI reduces code redundancy and optimizes development efficiency. Finally, Climate Action (SDG 13) is addressed through the environmental benefits of efficient AI-powered development, which can reduce computational overhead and lower carbon emissions in digital infrastructure (United Nations, n.d).

1.6 Scope and Limitations

This study follows a mixed method approach, incorporating both primary and secondary data collection. Primary data will be gathered through google Forms, utilizing a combination of open ended and closed ended questions. The data collection period is scheduled from August to November 2024, targeting participants involved in the Software Development Life Cycle in Finland and abroad. For example, developers, testers and project managers excluding parties not involved in SDLC process. Secondary data will be sourced from relevant studies and research related to this thesis, with Strict exclusion of any materials containing biased information. The type of studies that would be included would be industrial reports, academic research and research papers. The author will use concurrent triangulation design to minimize biasness in this research paper.

1.7 Thesis Structure

This thesis consists of six main chapters, each addressing a specific aspect of the research process. The first chapter introduces the study by presenting the background of the research, identifying the problem statement, and outlining the scope and limitations. In addition, it briefly describes the role of this research in sustainable development and the structure of the entire thesis to guide the reader through the content.

Chapter two presents the theoretical framework supporting the study. It introduces key theories such as the Diffusion of Innovation Theory and the Task-Technology Fit Model, which form the conceptual foundation for analysing the adoption and impact of generative artificial intelligence (GenAI) in software development. It also contains literature review, which examines existing studies and developments related to the Software Development Life Cycle (SDLC) and artificial intelligence. It explores the historical context of both SDLC and AI and discusses how GenAI is currently being utilised across various phases of the software development process.

Chapter three focuses on the research implementation. It outlines the adopted research methodologies. Both qualitative and quantitative research approaches are presented to support a comprehensive understanding of the topic. It explains the data collection process. It describes the design and distribution of the survey used as the primary data collection instrument and provides information on the target population and participants. It also explains the methods and procedures used for data analysis. This chapter describes how the collected data were processed and interpreted to generate meaningful insights aligned with the research objectives.

Chapter four presents the study's findings. It summarises key survey results, including the current use of GenAI in SDLC phases, its perceived benefits and challenges, and developers' perceptions and experiences. It also discusses anticipated trends in the use of generative AI in software development. Chapter five discusses the findings in the context of the theoretical framework and existing literature. It reflects on ethical considerations and methodological reliability and provides recommendations for future research.

Finally, chapter six concludes the thesis by answering the research questions. The appendices include the full version of the survey questionnaire and detailed survey results, which support the analysis presented in the thesis.

2 Literature Review

2.1 History of Software Development Life Cycle

In the early 1950`s, the software development process was not in existences as code development was highly influenced by outside forces due to limited hardware resources. Operating system were primitive and non-interactive but continued to grow up for the first computer aided software engineering tools which aided compiling, debugging and editing. There was a risk due to lack of software development methods (del Águila, Palma, & Túnez, 2014).

In the 1960`s, the first software crisis occurred. During this crisis period software engineers experienced recuring system development problems which caused the entire project to be delayed or over budgeted, non-responsive to the user and/or customer requirements, and difficult to use, maintain, and enhance (Gibbs,1994).

This crisis led to the birth of software development process. Structured methods, such as the Software Requirements Engineering Methodology (SREM) and the Structured Analysis and Design Technique (SADT), were innovated (Del Águila et al., 2014).

According to Gurang, Shah, and Jaiswal (2020), The oldest model used in software development is waterfall but not used much in recent years this model introduced and followed a linear-sequential lifecycle. It is straightforward to understand and implement. In Figure 1 illustrate In this model, each phase must be fully completed before transitioning to the next, with no overlap between phases.

The waterfall model was first documented by Benington in 1956, It was known as the cascade model. It was modified and edited by Winston Royce in 1970. It is known to be a backbone of all models with stages as shown in Figure 1. It was linearly created meaning that requirements need to be defined and analysed before moving to design or development (Ruparelia,2010).

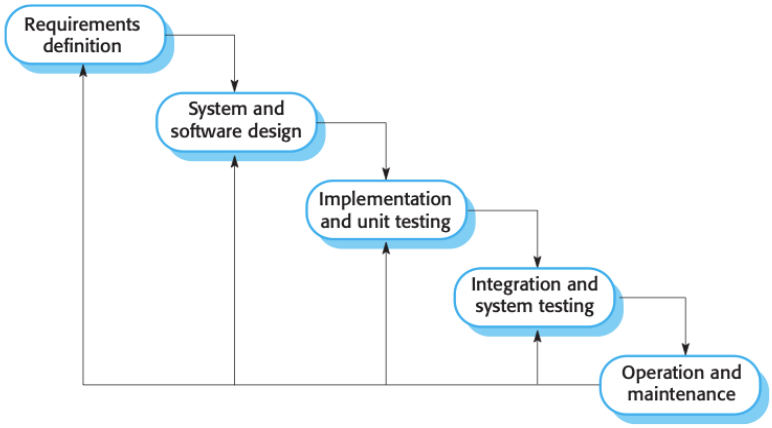

Figure 1. Representation of different phases of the waterfall model (adapted from Sommerville, 2016, p.47)

In 1988, the B-model was introduced as an enhancement to the waterfall model, incorporating maintenance as a fundamental part of the development process. This approach emphasized the continuous improvement of software or systems during development. Developers recognized the need to innovate alternatives to obsolescence, enabling newer systems to evolve as updates to the original. Later, the incremental model was introduced as a further improvement, allowing feedback from earlier phases to be integrated into the current phase. This model also involved stakeholders in identifying risks at an earlier stage (Ruparelia,2010).

In 1986, Paul rock introduced the V-model, a Software Development Life Cycle (SDLC) model characterized by sequential, V-shaped structure (Durmuş, Ustoglu, Tsarev, & Börcsök, 2018). Also known as the Verification and Validation (V&V) model, it extends the traditional waterfall approach. One of its defining features is the pairing of each development phase with a corresponding testing phase which is disciplined phase. It follows the strict step by step progression, requiring each phase to be completed before advancing to the next phase (khan,2023).

Barry Boehm introduced the spiral model in 1988. It merges the iterative approach of small, repeated cycles with a linear sequential flow of the waterfall model, with a strong emphasis on risk analysis. This model allows for gradual release and enhancement of software by developing prototypes at each stage of the process. By continuously refining the product through these

iterations, teams can effectively address the potential risk and make improvements throughout the development lifecycle (Amazon web Service, n.d).

In 2001, a group of software developers came together to publish the Agile Manifesto. The manifesto outlined key principles and values aimed at improving the software development process, emphasizing collaboration, flexibility, customer feedback, and iterative progress. The Agile method focuses on incremental development, emphasizing adaptability, teamwork, and customer input. Unlike traditional SDLC models like the Waterfall Model, which follows a strict linear sequence, the Agile SDLC breaks the development process into smaller, manageable iterations or increments, which are also known as sprints as shown in Figure 1. This approach allows teams to respond quickly to changes and incorporate feedback throughout the development cycle, resulting in more effective and user-centred software. The Agile approach has since evolved into various methodologies, such as Scrum, Kanban, and Extreme Programming (XP), each promoting Agile principles in different ways (Geeks for Geeks, 2024).

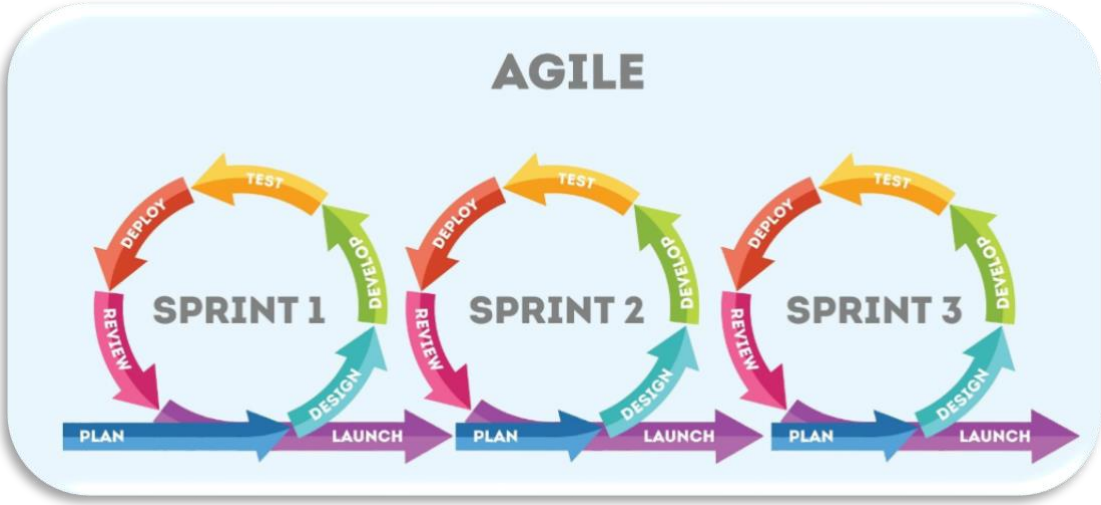

Figure 2. Agile model approach with sprints (adapted from SolDevelo, 2020)

2.2 History of Artificial Intelligence.

The field of artificial intelligence has grown into one with huge global influence. The definition of AI and its boundaries have shifted over time. Experts in the field often joke that AI encompasses all that computer currently can't do. While this may be light-hearted, there is an underlining truth to it (Bartneck, Lütge, Wagner, & Welsh, 2021).

The earliest AI model was inspired by the human brain's architecture, and it focused on simulating the functions of a single neuron. These models used straight forward, simple input-output functions. Over the decades AI systems have become more sophisticated, with more functions, layers and bidirectional feedback. These improvements have led to modern day deep neural networks. The first AI publication was done by McCulloch and Pitts in 1943. Their model was referred to as MCP neuron which describes a computer model that learns based on a process which is comparable with human brain. It worked in by taking Boolean inputs and processing them in present manner which MCP neuron would output a value (Muthukrishnan, Maleki, Ovens, Reinhold, Forghani, & Forghani, 2020).

In an improved MCP called the perceptron was published by Rosenblatt it was known to be flexible by processing non-Boolean inputs and including weights. This is known to be the building blocks for modern neural networks. Over the course of years, methods of evaluating these models have been developed. For example, Turing test in the 1950s which a blinded interrogator was tasked to question a human and a machine respondent to identify which among them was a machine. It is said that the machine passed the test (Muthurkrishnan et al.,2020).

In the 1960s the first example of Generative AI called ELIZA was created. It is one of the earliest chatbots created by Joseph Weizenbaum. It responded to humans using natural language and responses were designed to be empathic. In the 1970`s AnnB. Lesk, Leon D. Harmon and A.J: Goldstein developed some basics of recognition patterns which significantly increased its accuracy. In 1970´s Seppo Linnainmaa developed a modern version of backpropagation which is a process of propagating errors backward as part of learning process (Foote,2024).

In the 2000`s there was an enormous growth in the AI field. From 2004 to 2006, the Face Recognition Grand Challenge, funded by the U.S. government, led to significant improvements in

facial recognition technology. In 2011, Siri's launch on the iPhone 4S marked a surge in the use of virtual assistants. Another leap occurred in 2014 with generative adversarial networks (GANs) were introduced, enabling AI to create highly realistic synthetic data.

In the past few years, AI has grown rapidly and made big changes in technology. OpenAI launched GPT-3 in 2020, which became famous for creating text that sounds natural and human-like. By 2022, OpenAI introduced ChatGPT, making AI tools more user-friendly and popular for chatting, writing, and other tasks (Foote, 2024).

In 2021, GitHub Copilot changed how developers work by giving smart suggestions for coding, which helped save time and made coding easier. Generative AI also improved in creative fields—tools like DALL·E started creating very realistic images and artwork. Together, these AI tools and innovations have made industries more advanced and brought new ways to solve problems and work more efficiently (Foote,2024).

2.3 Software Development Life Cycle

Software development lifecycle is a model that defines an order of sequence of activities and phases that should or will take place in or during software development. It covers all stages from requirement identification stage through to deployment and maintenance (Mishra,2013).

Application Lifecycle Management (ALM) is the integration of people, processes, and tools to manage an application's lifecycle, from conception through development, deployment, and maintenance, until its eventual retirement. ALM emphasizes governance, resource management, and continuous delivery of updates, aligning applications with organizational goals while ensuring sustainability and adaptability over time (Red Hat, 2020).

Software Development Life Cycle (SDLC) is a crucial component of Application Lifecycle Management (ALM), focusing on the stages of software development such as planning, designing, coding, testing, and deployment. While SDLC addresses the creation and delivery of software, ALM provides a broader framework that encompasses the entire lifespan of an application. By including SDLC within its structure, ALM promotes efficient collaboration and visibility across all

stages. It enables iterative methodologies like agile and DevOps, fostering frequent releases and seamless integration of processes (Red Hat, 2020).

The introduction of SDLC is traced down to Dawson and Dawson through Benington Heralds who wrote the importance of SDLC. In 1956 the first software process model was introduced by Benington. This model has been recognized by software developers throughout the years and has formed the foundation in which all existing SDLC models used (Sarvanos & Curinga,2023). Examples of SDLC models are, Waterfall, Spiral, RAD, incremental, V-shaped and Agile methods (Mishra,2013).

2.4 Generative Artificial Intelligence in Software development Phases

Generative Artificial intelligence (AI) refers to system capable of producing various types of content. Including text, audio, images, and code. These models are trained to recognize patterns and structure within their datasets, enabling them to generate new content with similar characteristics. A subset of Generative AI, Large language Models (LLMS), excels in natural language tasks such as text generation, translation, summarization, and question answering (Dakhel, Nikanjam, Khomh, Desmarais, Washizaki, & Abrahamsson, 2024).

Notable LLMs include OpenAI's GPT series (e.g., GPT-3.5, GPT-4), Google's PaLM, and Meta's LLaMA. These models have also demonstrated significant potential in software development task, including code generation, debugging, and test case generation. This showcases the growing applicability of LLMS in software engineering, where they contribute across various phases of the software development lifecycle, such as requirement gathering, design, development, testing, and maintenance (Dakhel et al., 2024).

The architecture of LLMs primarily relies on the transformer model, which is a deep learning architecture designed to process sequential data. Key components of the transformer architecture include Tokenization, Attention mechanisms, Encoder-Decoder structure Activation functions and prompt. In addition to core architecture, there are various methods for adapting LLMs to a specific task which include pre training, fine-tuning, context learning and augmentation (Dakhel et al., 2024).

Large Language models (LLMs) such as GPT-3 and Codex have emerged as powerful tools for automating software engineering tasks, including code generation, debugging, code summarization, and test case generation. While LLMs have shown promise in these areas, their application in software engineering is still evolving, and several challenges remain. These challenges include the need for high- quality datasets for finetuning, the high computational cost of training and adapting LLMs, and the risk of generating hallucinations, plausible but incorrect outputs. To address these challenges, various advanced techniques and frameworks have been developed to augment LLMs with external knowledge, tools, and reasoning capabilities, improving their ability to perform software development tasks (Dakhel et al., 2024).

Artificial intelligence has influenced and transformed SDLC process. Inclusion of Artificial intelligence in SDLC process has dramatically changed the way software is developed, tested, deployed and maintained. Artificial intelligence in software development is about automating route operations, influence on decision making and improving productivity (Kumar,2024).

2.4.1 Artificial Intelligence in Requirements Identification and Gathering

In this phase, a software, functional and user requirement document is created to collect relevant information from the client about the development of the product as per their expectations. In this phase, system input, process and interfaces are illustrated in detail and all ambiguities are solved (Prakash,2024).

Artificial intelligence technologies significantly enhance the phase by automating the identification, analysis, and extraction of requirements. Utilizing methods such as data mining and pattern recognition, AI systems can process large volumes of textual and contextual data to extract relevant requirement from sources like user feedback, documentation, and stakeholder interviews. By automating labour intensive task, Artificial intelligence accelerates the requirements elicitation process minimizes manual errors and ensures the completeness and accuracy of the gathered requirements (Smith & Wei,2024).

Natural Language Processing (NLP) is an asset in the requirement and specification phase. It enables computers to understand, interpret, and generate human language, facilitating effective communication between software systems and stakeholders. NLP-powered systems can analyse

natural language inputs from stakeholders, extract key concepts, and refine them into structured requirements specifications. By leveraging NLP, organizations bridge the gap between domain experts and technical teams, fostering collaboration throughout the requirements gathering process (Smith & Wei,2024).

2.4.2 Artificial Intelligence in Design and Architecture

In this second phase, the requirement identification document is used as an input for software design and architecture. It assists in specifying hardware and systems requirements. Designs include database and use interface designs. The output is usually the flowcharts, architecture of the system and sequence diagrams (Parkash,2024).

Artificial intelligence provides an alternative approach to creating prototypes and architectural models during the design phase of the Software Development Life Cycle. AI-powered systems allow designers to explore a wide range of innovative design alternative, optimize performance, uncover solutions that may not be immediately apparent using traditional design methods. In this phase AI can enhance creativity and innovation, ultimately leading to the creation of more efficient and well-structured software architectures (Smith & Wei,2024).

2.4.3 Artificial Intelligence in Development and Programming

In this phase, design is converted to functional systems. Designs are converted into source code using programming tools or languages (Prakash,2024). The actual code is written based on design and specifications. The developer ensures that individual component work as intended and code modules are interacted as needed (Mohamed,2023).

AI-driven programming assistants help developers write cleaner, more maintainable code while minimizing errors. Using techniques like predictive modelling and pattern recognition, AI -powered refactoring tools recommend code transformation and optimizations that adhere to best practices and programming standards. These tools not only speed up the refactoring process but also assist developers in producing more efficient code, improving software quality and reducing technical debt. Additionally, automated bug detection and resolution is another crucial application of AI in the development phase of the SDLC (Smith & Wei,2024).

2.4.4 Artificial Intelligence in Testing

In this phase the developer determines if the requirement identified in the document are met. There are different levels of testing, unit level, integration level and system level testing. Unit level is done just after development phase. It involves testing small portions of the code. In integration testing level different units, modules or components are tested as a combined entity. Several intergraded software modules may have dependencies phase. It involves testing to verify if all the user level requirements have been met (Prakash,2024).

AI-powered regression testing tools, anomaly detection techniques and testing frameworks assist the software developers to detect and solve issues early in the project development process. This reduces the risk of software failures and improves overall quality and reliability. AI-driven test case generation tools easily identify critical paths, edge cases, and input combinations' that boost test coverage present possible software defects (Smith & Wei,2024).

2.4.5 Artificial Intelligence in Deployment and Maintenance

In this phase the software is installed in customers environment. The product may also be released to the market. The product is maintained in that, latest errors are fixed and updated. Potential enhancements are identified to address any potential future user end problems (Prakash,2024).

The aim of this deployment phase is to make sure that a smooth transition from development to production environment is achieved. In this stage any remaining bugs are fixed to ensure that the software works as intended. There are various deployment methodologies some are continues deployment (CD) which involves code changes that pass the automated testing and is automatically released to production. Rolling deployment is another approach which allows a system to be updated incrementally over time. It follows a stage role out where the new version of the application is gradually deployed, preventing down time this are the few mentioned deployment strategies (Van Pham, 2024).

AI plays an important role in keeping systems reliable and efficient during deployment and maintenance. Predictive maintenance uses AI to analyse data and detect potential issues before

they occur. By monitoring system logs, resource usage, network traffic, and user activity, AI can identify unusual patterns that may signal future problems. Machine learning techniques, such as time-series analysis and anomaly detection, help recognize warning signs of system failures. This proactive approach reduces downtime, lowers maintenance costs, and improves overall system performance (Ale nezi & Akour, 2025).

AI also improves automated monitoring and alert systems by detecting problems in deployed applications. AI-powered alerts can differentiate between normal system activity and actual issues, reducing false alarms and helping teams focus on real problems. When integrated into CI/CD pipelines, AI can automate post-deployment monitoring by analysing system logs, detecting anomalies, and notifying developers of any issues. AI can also create unit tests during deployment, improving test coverage and software reliability (Ale nezi & Akour, 2025).

Additionally, AI can monitor system logs after deployment to catch problems early, preventing them from affecting users. Advanced AI methods, such as machine learning and natural language processing, can quickly analyse large amounts of data to find hidden issues. Automating these processes reduces manual work and allows teams to respond to problems faster, making the system more efficient and reliable (Ale nezi & Akour, 2025).

2.5 Impact of Generative AI in Software Development Process

There are various industry reports, research papers and academic articles aimed at researching on the effect of generative AI in software development process.

McKinsey & Company (2023) study report show that software developers can complete task twice as fast with use of generative AI. Its empirical research show that AI-based tools are revolutionizing the workflow and boosting efficiency like never before. This research shows generative AI-based tools significantly boost productivity by hastening routine tasks, jump-starting new code, accelerated updates, and enhancing developers' ability to tackle new challenges.

In the coding phase of software development process, writing new code can be accomplished nearly twice as fast, and optimizing and refactoring existing code take roughly two-thirds of the original time. Moreover, documenting code for maintainability task is cut into half. According to

this report, generative AI-tools and processes lead to productivity improvements that surpass previous advancement with proper training and support within organizations (McKinsey & Company, 2023).

According to McKinsey & Company (2023), despite potential productivity increase with generative AI-tools, time savings vary based on task complexity and developer experience. Task deemed highly complex when tackled by junior developers sometimes show minimal gains or even increased time. Quality, However, remained stable or improved with AI collaboration. To optimize productivity and maintain code quality a structured approach incorporating AI training, appropriate use cases, workforce development, and risk management is essential. This structured approach ensures effective deployment and maximizes the benefits of generative AI in software development.

A study done by McKinsey & company in 2024 shows that organizations started deriving significant business value from generative AI, with 65% of respondents in a McKinsey survey indicating regular use of the technology almost double from the previous survey. Expectations for generative AI's impact remain high, with three-quarters predicting significant or disruptive changes in their industries in the coming years. Figure 3 presents an increase in adoption of generative Artificial intelligence from 2017 to 2024 (McKinsey & Company, 2024).

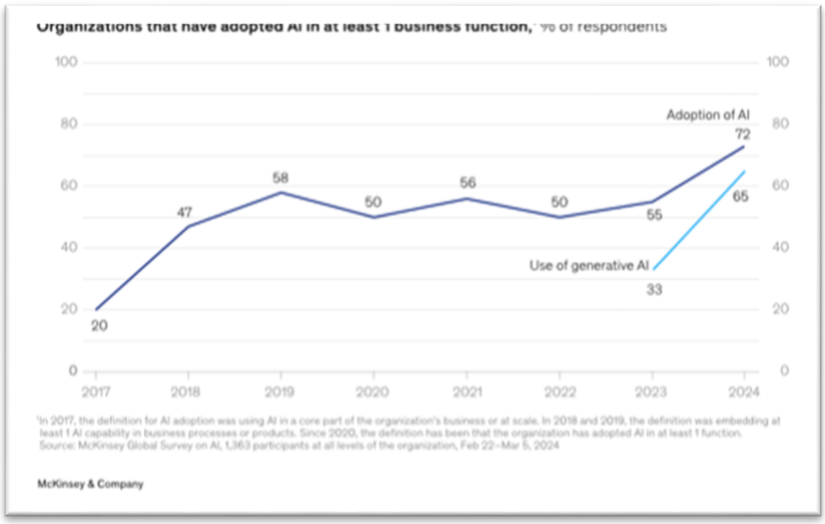

Figure 3. Adoption of Artificial intelligence (McKinsey & Company, 2024).

A peer-reviewed study was done in 2023 and posted in IJCRT organization. This study was conducted to understand the implications of integrating Generative Artificial Intelligence into software Development Life Cycle. The study involved purposive sampling to select a pool of 30 participates, primarily software engineers, UX designers, and project managers from diverse organizations. The average work experience of the participants was 12.6 years. Interviews and surveys were conducted in July 2023, each lasting 45-60 minutes, with data transcribed and anonymized for analysis (Pothukuchi, Kota, & Mallikarjunaradhya, 2023).

The results show a significant impact of Generative artificial intelligence in various stages of the SDLC. These impacts were speed: over half of the respondents believe that the pace of software deployment will significantly accelerate within the next five years due to Generative Artificial Intelligence. Proliferation: 80% of respondents reported using Generative Artificial Intelligence in their daily tasks, indicating its widespread adoption in the tech industry. Reliability: 60% of respondent anticipate that by 2028, Generative artificial intelligence will be reliable enough for generating production-quality code and technical documents. Workforce Impact: 50% of respondents expect a reduction in workforce requirements due to increased reliance on Generative Artificial intelligence, with many technical roles potentially shifting to role of reviewers. The study identified several limitations of Artificial intelligence, including biases, inaccuracies, lack of coherence, originality issues, emphasizing the need for human oversight (Pothukuchi et al., 2023).

Developer survey report done by stack overflow (2024) explores the use, perceptions and effects of Generative Artificial intelligence tools focusing on their integration into the development process, benefits, challenges, and ethical concerns. Over 60,000 people participated in the study.

The study reported a steady increase in adoption of Generative Artificial Intelligence. 76% of respondent are using or planning to use AI tools this year (2024) which has increased from 70% last year. The actual usage has also increased from 44% to 62%. Developers mostly use Artificial intelligence tools for writing code (82%). Those not yet using Artificial intelligence tools are mainly curious about testing code (46%). 72% of respondents view Artificial intelligence favourably, though this is slight decline from 77% last year (2023), possibly due to disappointing results for example handling complex tasks as shown in Figure 4 below. (Stack Overflow, 2024). (see Figure 4)

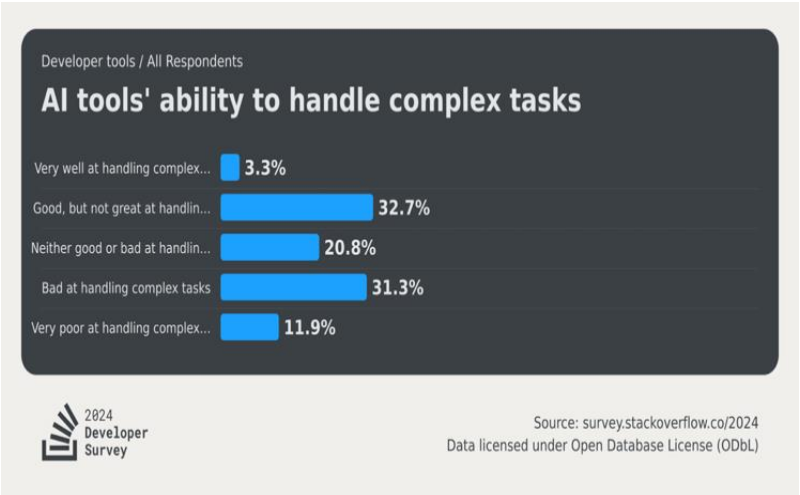

Figure 4. AI tool’s ability to handle complex tasks (Stack Overflow, 2024).

In Figure 5 below, 81% of respondents agree that increasing productivity is the biggest benefit of Artificial Intelligence tools. AI is seen as more beneficial for speeding up learning among new programmers (71%) compared to professional developers (61%). 43 % of developers trust AI output, while 31% are sceptical. New programmers trust AI more than professionals (49% vs. 42%). 45% of professional developers believe AI tools are poor at handling complex tasks (Stack Overflow, 2024).

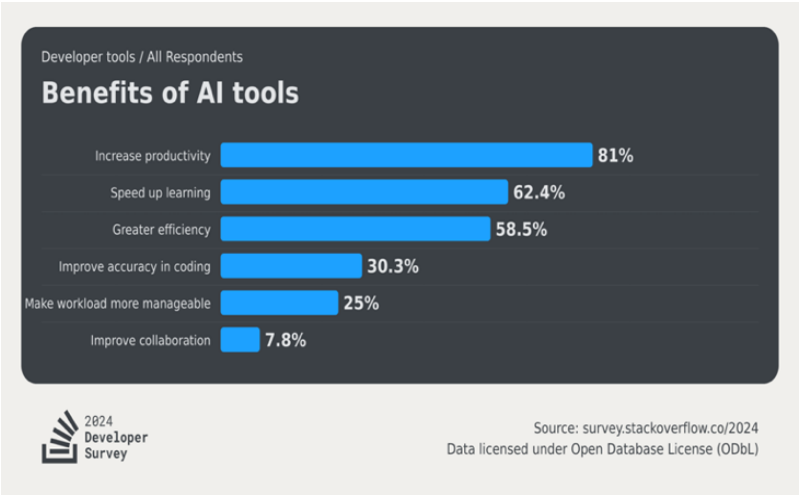

Figure 5. Benefits of AI tools (Stack Overflow, 2024).

In the coming Year, Developers expect AI tools to be integrated into documenting code (81%), testing code (80%), and writing code (76%). 70% of professional developers do not see AI as a threat to their jobs. Misinformation (79%) and source attribution (65%) are the top ethical concerns. Developers also highlight trust and understanding of the code base as key challenges (Stack Overflow, 2024).

A survey commissioned by GitHub and carried out by Wakefield Research was conducted online between March 14 and March 29,2023. It included 500 U.S.-based developers who are not students, who do not hold managerial positions, and work for companies with over 1,000 employees. According to the survey, AI coding tools are well integrating into existing developers' workflows, enhancing efficiency without fundamentally changing the software development lifecycle. Developers recognize these tools as productivity boosters, complementing long-standing automation practices like machine learning based security checks and CI/CD pipelines. Instead of restructuring process, AI tools streamline operations, allowing developers to focus more on creating innovative solutions (GitHub, 2025).

AI tools help streamline processes by supporting code quality, reducing errors and speeding up development, which, in turn, helps teams meet performance standards and foster collaboration. Developers believe that these tools can also play a significant role in upskilling by integrating learning opportunities directly into their work without adding extra tasks. Companies that adopt AI tools can expect benefits such as increased developer satisfaction, better code quality, faster

outputs, and improved team collaboration. For engineering leaders, leveraging generative AI can lead to more productive, collaborative, and satisfied development teams, ultimately enhancing the software development life cycle (GitHub, 2025).

The virtual round table was conducted by Tracy (Trac) Bannon from MITRE Corporation and Phil Laplante, a computer scientist at NIST and professor emeritus at Penn state University. The discussion was part of the ArchAITecture Research Collaborative, which focused on the intersection of AI and software engineering (Bannon & Laplante, 2024).

In the roundtable, it was highlighted that GenAI supports developers by automating repetitive tasks, improving coding speed, and enhancing the efficiency of the development process. Allowing teams to focus on more complex and value-driven tasks. GenAI shifts the focus of developers from manual coding to strategic oversight, problem-solving and collaborations (Bannon & Laplante, 2024).

Trust, security, and transparency remain a significant concern. Experts highlighted risk such as data privacy issues, potential code inaccuracies, and the long-term implication of reduced human engagement in routine coding tasks. The roundtable raised question about trust and transparency, especially with the low level of trust in AI-generated code (Bannon & Laplante, 2024).

The experts suggest that companies should create metrics that not only capture immediate benefits of GenAI but also align with long term strategic objectives. Tracking this outcome helps measure the broader impact of generative AI on the SDLC For example, metrics related to the reliability and security of AI outputs, such as errors rate in generated code or accuracy in test results, are crucial for assessing GenAI effectiveness (Bannon & Laplante, 2024).

The roundtable concluded that while GenAI can enhance SDLC processes, it must be adopted thoughtfully to avoid issues like deskilling and maintaining human innovation. Long-term success depends on maintaining a balance between automation and human creativity. Developing robust security practices and ensuring AI integration aligns with company cultures (Bannon & Laplante, 2024).

A study was conducted by Ulfsnes, Moe, Stray, & Skarpen (2024) supported by the Research Council of Norway, to explore transformative effects Of Generative AI tools, such as ChatGPT and GitHub Copilot, on software development practice. The researchers conducted a qualitative interview with software developers, data scientist, and managers from multiple organizations. The interviewers were selected through snowball sampling, ensuring a diverse range of experience and perspective from both technical and leadership roles. The study found that GenAI tools were used for a variety of tasks, including code generation, code review and refactoring, automating documentation, testing and debugging and virtual environment for example simulating real-world scenarios for training.

The information reported that the use of GenAI tools significantly improved productivity, enabling them to spend less time on repetitive task and more on creative and complex work. The tools provided quick feedback, which was perceived as both time-saving and motivational contributing to a more enjoyable work environment. GenAI was described as less formal, and requiring no social consideration, making it more accessible tool for quick queries and problem solving (Ulfsnes et al., 2024).

Despite the benefits, the study also highlighted several challenges which include, the absence of seamless integration GenAI tools and other parts of the software development workflow let to a substantial amount of copy pasting between tools. While GenAI is effective at generating content quickly, the output often required additional modification and validation before it could be applied in practice. The quality of the output was heavily dependent on the quality of the prompt given to the tool. Developer who used prompt engineering techniques, such as contextualizing the problem or assigning roles to the AI found that they could achieve more precise and relevant results. Developers were cautious about inputting sensitive or proprietary information into the tools, requiring the cleansing and anonymization of data before use (Ulfsnes et al., 2024).

2.6 Theoretical Framework

This study draws on two primary theoretical perspectives to frame the integration and effectiveness of Generative Artificial Intelligence within the Software Development Life Cycle (SDLC). The first is the Diffusion of Innovation (DOI) theory, which explains how

innovations are adopted across different user groups over time, emphasizing communication channels, social systems, and time as critical factors in the adoption process. The second is the Task-Technology Fit (TTF) model, which focuses on the degree to which a technology supports a user in carrying out their tasks effectively, linking successful technology use to a strong alignment between the tool and the task.

The Diffusion of Innovation (DOI) theory, first introduced by Gabriel Tarde in 1903 and later developed by Everett Rogers, explains how new ideas, technologies, or practices are accepted over time by people in a group or society. Adoption usually begins with a small number of innovators. These early users encourage others (early adopters), then the early majority, late majority, and finally, laggards until most people have accepted the new idea (Kaminski,2011).

This theory highlights the need to adapt new ideas or technologies to suit the different groups of adopters. It also emphasizes the role of communication and peer networks, like talking with others or using social media, in spreading the innovation. Innovations are more likely to succeed when they are easy to understand, useful, compatible with people's needs, can be tried out, and are visible to others (Kaminski,2011).

Rogers outlined five steps in the adoption process: Becoming aware of the idea (awareness), Showing interest and seeking more details (interest), Deciding whether it will be helpful (evaluation), Trying it out (trial), Fully adopting and using it (confirmation) (Kaminski,2011).

The theory also includes the idea of "re-invention," which means adjusting the innovation to better suit different groups. Time and social interactions play key roles in how quickly and widely the innovation is adopted (Kaminski,2011).

This theory provides a useful way to understand how new technologies, like Generative AI, can spread and gain acceptance across different groups. It shows the importance of communication, peer influence, and making the innovation easy to adopt.

The second theory is The Task-Technology Fit (TTF) Model. It was created by Goodhue and Thompson in 1995, focusing on how well technology aligns with the tasks users need to perform

and how this alignment affects their performance. Unlike earlier models which focused on how people accept and adopt to the technology and their intentions to use it, the TTF model looks at what happens after adoption. It explores how technology use impacts individual and organizational performance (Marikyan & Papagiannidis, 2023).

The TTF model introduces the concept of "Task-Technology Fit," which refers to how well a technology's features meet the specific requirements of a user's tasks. It also presents the Technology-to-Performance Chain (TPC), which consists of three elements: task-technology fit, utilization, and performance impact. Task-technology fit is influenced by how well the technology matches the user’s skills and task needs. Utilization refers to how often and in what ways the technology is used, and performance impact measures the outcomes, such as increased productivity or better decision-making (Marikyan & Papagiannidis, 2023).

The TTF model suggests that the success of technology is not just about how widely it's accepted, but how well it addresses the user’s specific needs. Studies have shown that when there’s a poor match between the task and the technology, it can negatively affect productivity, even if the technology is being used regularly. Thus, the TTF model helps to bridge the gap between technology adoption and its actual effectiveness in improving performance (Marikyan & Papagiannidis, 2023).

This model is especially useful when assessing how well tools like Generative AI fit into software development. By examining how these tools align with the tasks in the Software Development Life Cycle (SDLC), the TTF model emphasizes the importance of ensuring that technology is designed to meet user needs, thereby boosting performance and efficiency (Marikyan & Papagiannidis, 2023).

3 Research Implementation

3.1 Research Method

This thesis aims to explore the impact of Generative Artificial intelligence in Software Development life cycle. Mixed method research approach is used for this thesis as it combines the strengths of both qualitative and quantitative methodologies.

This thesis adopts concurrent triangulation design which means two or more methods are used simultaneously to confirm, cross validate or corroborate findings within a study. Data collection occurs concurrently, allowing for the strengths of one method to address the limitations of the other. This design helps mitigate biases and enhances the reliability of the findings (Gunasekare, 2015).

The incorporation of artificial intelligence into the software development life cycle is a relatively new concept with limited prior research. Furthermore, many software development companies have yet to extensively adopt Artificial intelligence (Wilkinson, 2023). This presents an evident research gap. By employing a mixed methods approach with concurrent triangulation design, this thesis aims to provide a balanced and comprehensive understanding of the topic.

3.2 Data Collection

The author collected both primary and secondary data using a qualitative approach. Primary data was gathered through open-ended survey questions, allowing participants to express their opinions and perspectives in their own words. Secondary data means existing data collected by other parties such as government institutions, industrial institutions, individuals and or academic institutions and analysed by them respectively (Silva Martins, Carneiro da Cunha, & Serra, 2018). This research incorporates secondary data by synthesizing findings from industrial reports and previous academic research. Relevant search terms, filters and inclusions /exclusion criteria was utilized during data search.

Literature review served as an initial exploration of the existing body of knowledge surrounding a research topic. It involves identifying and analysing relevant sources to build through

understanding of the subject. By conducting this review, the researcher was able to uncover existing theories, identify knowledge gaps, and establish a strong foundation for the research (Abreu, 2023).

Quantitative research is a systematic approach that focuses on collecting and analysing numerical data to understand various phenomena. It emphasizes the importance of using numerical data to gain insights and form decision making. Quantitative research typically includes the collection of numerical data through surveys, experiments, and other instruments. The data is then analysed using statistical techniques to summarize findings and draw relevant conclusions (Alek, n.d).

The author uses online surveys to collect data by use of Google Forms and uses mixed method to analyse. Survey research is a method used to gather information to answer specific questions, solve problems, assess needs and evaluate objectives. The data is collected from individuals, making it subjective, and findings are typically generalized from a selected sample back to the larger population (Glasow, 2005).

Data collection took place between August 2024 and November 2024.Both primary and secondary data were collected concurrently to fulfil the research objectives. The author collected primary data using google form due to its accessibility ease of use, and cost effectiveness. This platform allows researchers to reach a diverse audience facilitating higher response rates while providing a user-friendly experience for both the author and respondents. Google Forms enables real-time data collection which is beneficial for quickly assessing responses and making necessary adjustments (Jotform Editorial Team, 2024).

The survey was intended to gather responses from 100 participants worldwide, focusing on individuals involved in the software development process, such as developers and project managers. To achieve this, the author employed a combination of purposive, convenience, and snowball sampling methods. Purposive sampling was used to identify participants based on specific criteria aligned with the research objectives, ensuring valuable insights. Convenience sampling followed, selecting participants who were accessible and available to the author (Saumure & Given, 2008). Finally, snowball sampling was implemented, encouraging participants

to share the survey within their networks, particularly among software developers, to expand the reach of the study (McCombes, 2019).

Despite efforts to maximize participation, the survey yielded 25 responses, significantly lower than the anticipated target of 100 participants. This limited response rate may reduce the generalizability of the findings, yet the data collected provides valuable perspectives and serves as a foundation for further exploration. The participants' feedback highlights critical trends and challenges that inform the broader discussion of generative AI's role in software development. To support the analysis, secondary data was gathered from books, journals, websites, and articles to cross-validate the findings and strengthen the conclusions.

3.3 Data Analysis

The author used mixed method analysis with a concurrent triangulation design approach. This served as the backbone of the data analysis process, enabling the integration of qualitative and quantitative datasets to provide a balanced perspective. By analysing both datasets concurrently, this method helped validate and cross-check the findings, ensuring their credibility and reliability.

Qualitative data, which included open-ended survey responses and secondary data, was analysed using thematic analysis. This method allowed for the identification of recurring patterns and themes across the data. By categorizing responses and synthesizing secondary sources, thematic analysis provided a structured and insightful interpretation of the qualitative data. The themes identified were, adoption rate of GenAI in SDLC, positive and negative impacts of GenAI in SDLC, perceptions and experiences of software developers and future of AI in software development life cycle (Braun & Clarke, 2012).

Quantitative data, collected through closed-ended survey responses, was analysed using weighted averages. This technique offered a nuanced understanding of the numerical data by assigning relative importance to different response options. Weighted averages provided a clearer picture of trends and helped quantify data. It is usually employed to account for varying levels of significance among the different data points. Unlike a simple arithmetic mean, which treats all observations equally, the weighted average assigns different importance, or weight to each data point based on

predefined criteria. This makes the weighted average particularly useful when certain data points have a greater influence on the outcome than others (Frost, 2025).

The weighted average formular used is as follows

$$Weighted\ average = \sum(i = 1)^n(w_i x_i) / \sum(i = 1)^n w_i$$

X_i Represents the value of each data point,

∑ Denotes summation across all data points.

W_i Represents percentage of respondents for each category

In this research, X_i impacts score category is:

- 0 represents no impact
- 1 represents slight impact
- 2 represents moderate impact
- 3 represents significant impact
- 4 represents transformative impact

The integration of qualitative and quantitative methods through concurrent triangulation ensured that the strengths of one dataset addressed the limitations of the other. This approach enhanced the comprehensiveness of the analysis while mitigating biases. Additionally, visual representations such as graphs and charts were utilized to summarize and convey quantitative trends effectively.

Limitations, such as the lower-than-expected response rate, were addressed by leveraging secondary data to supplement and validate the primary findings. The concurrent triangulation approach played a key role in overcoming these challenges and maintaining the integrity of the analysis.

4 Results

4.1 Background and Demographic

Basic information about the respondents was collected. As shown in Figure 6, most respondents, were from Africa (36%) and 32% from north America and 28% from Europe and the least was Asia 4.0%.

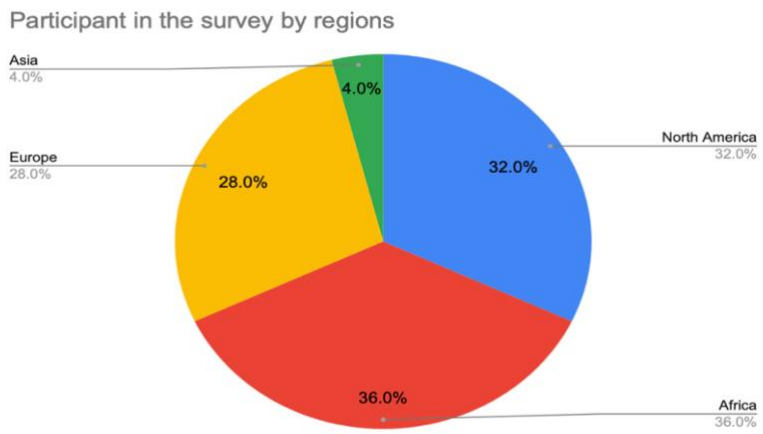

Figure 6. Participants in the survey by regions

As shown in Figure 7, data pertaining to respondents` experience level was collected. Most respondents (36%) had 1-4 years, 28% had 5-9 years of experience, 24% had less than a year, 8% had 15+ years' experience and 4% had 10-14 years in the field (see Figure 7).

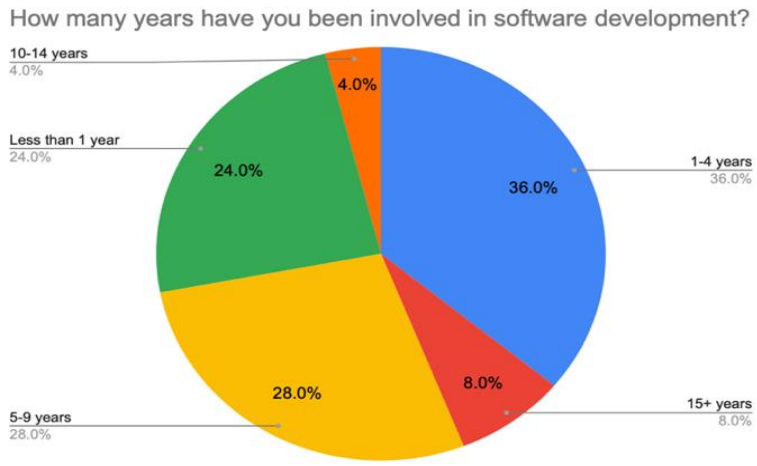

Figure 7. Experience of participants based on years

Figure 8 illustrated the use of Generative A tools I in software development. 40 % reported not to use while 60% reported to use generative AI in the software development process.

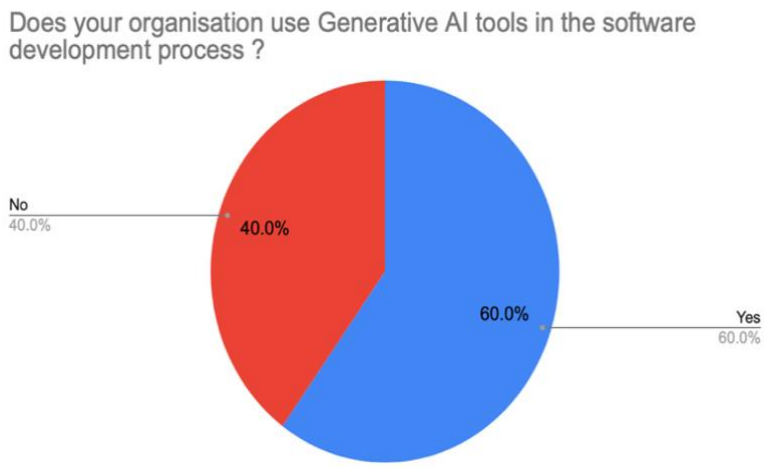

Figure 8. Usage of Generative AI in organisation

The findings in Figure 9 showed that most of the respondents are software developers, in which most responded using Agile Methodology (52%) as their organization's SDLC model.

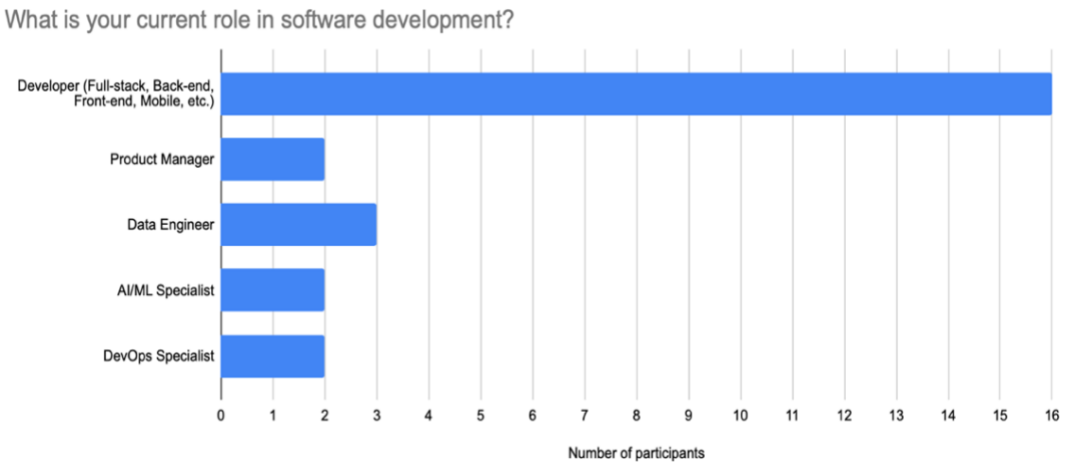

Figure 9. Current role of participant in software development

The findings in Figure 10 show that Agile methodology (52%) is the most commonly used SDLC model in the respondents' organizations.

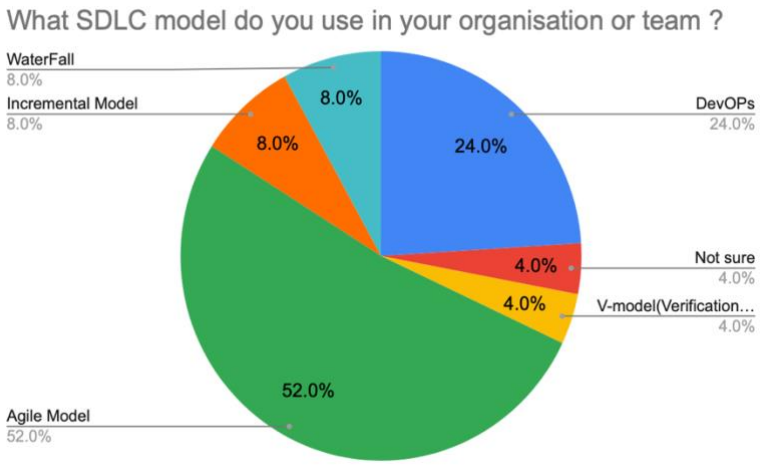

Figure 10. Software models used in organisations or teams

4.2 Adoption of GenAI in Software Development Life Cycle Phases.

As illustrated in Table 1. Calculated Weighted Average adoption scores in Software Development Life cycle phases from primary data., the highest score is in the coding and implementation phase,

with a weighted average of 1.92. This indicates that Generative AI has highly impacted or mostly implemented in writing code. The design has a weighted average of 1.68 and Testing having weighted average of 1.48 which shows a notable adoption. The lowest impact is seen in Requirement Gathering and analysis 0.95, meaning artificial intelligence is least useful in this phase. These findings are derived from primary data collected through a structured online survey, with the original response distributions illustrated through pie charts in Appendix 2 Figure 11-16).

Table 1. Calculated Weighted Average adoption scores in Software Development Life cycle phases from primary data.

SDLC Phase	**No Impact (0)**	**Slight (1)**	**Moderate (2)**	**Significant (3)**	**Transformative (4)**	**Weighted Avg. Score**
Requirement Gathering and Analysis	40%	28%	24%	8%	0%	0.95
Design	24%	28%	44%	0%	4%	1.68
Coding & Implementation	16%	16%	28%	40%	0%	1.92
Testing	20%	40%	20%	12%	8%	1.48
Deployment & Maintenance	30%	20%	28%	16%	0%	1.32

Secondary data as shown in Table 2. Secondary data collected from different research indicating where GenAI has being implemented in SDLC, from McKinsey & Company (2023), stack overflow (2024), and Ulfsnes et al. (2024) validate that, indeed, Generative Artificial intelligence has been highly adopted in coding followed by testing and maintaining code documentation. In the Coding phase, Artificial intelligence significantly speeds up coding tasks. Enabling faster delivery of software. However, developers face concerns over quality and complexity of Artificial intelligence

code. Some developers report that artificial intelligence struggles with more complex problems, leading to scepticism in its reliability. Secondary data shows that the most used GenAI tools in generating code is ChatGPT and Co-pilot.

Table 2. Secondary data collected from different research indicating where GenAI has being implemented in SDLC

Phase	Implementation/adoption	Study source	Example/Explanation
coding	AI tools like GitHub Copilot and ChatGPT enhance code generation, refactoring, and quality.	McKinsey (2023), Stack Overflow (2024), Ulfsnes et al. (2024) (Bannon & Laplante, 2024).	AI tools speed up coding processes by up to 2x. Developers can focus more on creative problem-solving rather than manual coding.
Testing & debugging	AI automates error detection, routine tests, and suggests potential fixes.	GitHub (2023)	AI tools help automate regression testing, identify bugs quicker, and recommend fixes to common errors.
Maintaining Documentation	AI tools assist in automatically generating and maintaining code documentation.	McKinsey (2023)	AI can produce code comments and document functions, saving time and reducing manual documentation workload.
Collaboration	AI tools facilitate communication, provide feedback, and enable real-time collaboration among teams.	GitHub (2023)	Developers spend less time debugging and more time discussing solutions and innovating due to AI-enhanced collaboration tools.

4.3 Positive and Negative Impacts of Generative AI on SDLC

Table 3 below shows that while Generative Artificial intelligence is improving efficiency (weighted Av =2.00) and productivity (weighted Av = 2.08) in software development, the overall impact is moderate. Developers report a lot of challenges (Total negative = 5.16) and benefits (Total positive = 7.24) suggesting that, while developers benefit from GenAI, negative challenges are still of a concern. The biggest challenges to using generative artificial intelligence are issues with data

quality (weighted Av = 3.0), integrating it with existing tools, and concerns about ethics. These problems suggest that while generative artificial intelligence has potential, more work is needed to address its limitations and ensure it is used responsibly in businesses. These findings are derived from primary data collected through a structured online survey, with the original response distributions illustrated through pie charts in Appendix 2 (Figure 17-21).

Table 3. Survey showing the positive and negative impacts in SDLC from primary data

Category	Breakdown (Percentage)	Weighted Average Score
Contribution to Efficiency	40% Moderate (2), 24% Significant (3), 16% Minor (1), 12% No (0), 8% Major (4)	2.00
Improvement to Productivity	32% Moderate (2), 32% Significant (3), 16% Minor (1), 12% No (0), 8% Major (4)	2.08
Enhancement to decision making	44% Moderate (2), 16% Slight (1), 12% Significant (3), 4% Extremely (4), 24% No (0)	1.56
innovation	44% Moderate (2), 24% Slight (1), 16% Significant (3), 12% No (0)	1.60
challenges	52% Data Quality (4), 36% Integration (3), 32% Ethical (2), 20% Skilled Personnel (1)	3.00
Ethical implications	60% Moderately Concerned (2), 32% Very Concerned (3), 8% Not Concerned (0)	2.16

Secondary data validates and corroborates primary findings. While Table 4 highlights the positive advancements of generative AI such as increased development speed, Table 5 reveals persistent concerns, including trustworthiness, job displacement, and ethical implications. Despite the benefits, developers remain sceptical about AI's ability to handle complex and nuanced tasks. Additionally, there are significant concerns related to security risks, such as biases in AI-generated code and potential misuse of sensitive data.

Table 4. Positive impacts data from secondary sources

Positive Impact	Study source	Example/Explanation
Efficiency and Speed	McKinsey (2023)	Tasks like code generation, testing, and documentation can be completed faster, boosting productivity and reducing delivery times.
Improved Learning for Juniors	Stack Overflow (2024)	Junior developers can learn quicker with AI-assisted coding and problem-solving tools, making complex code more understandable.
Increased Collaboration	GitHub (2023)	AI enables quicker feedback and smoother collaboration among developers, making teamwork more effective and innovation driven.

Table 5. Negative impact data obtain from secondary source

Negative Impact	Study Source	Example/Explanation
Quality and Trust Issues	Stack Overflow (2024), IJCRT (2023)	AI tools are not always reliable for handling complex tasks, and developers often hesitate to fully trust AI-generated code for production-level tasks.
Job Displacement Concerns	IJCRT (2023), McKinsey (2024)	AI could potentially replace junior developers, manual testers, and other roles that involve routine tasks, leading to job reductions in specific areas.
Ethical and Security Concerns	MITRE/NIST (2024), Stack Overflow (2024)	Ethical issues such as AI bias, data privacy, and security concerns (e.g., AI tools using proprietary data) need addressing to ensure the responsible use of AI.

4.4 Perceptions and Experiences of Developers Regarding Generative AI

The perception and experiences of developers using generative artificial intelligence tools are varied, with some embracing the technology and others remaining cautious. This section outlines the key findings regarding developers' sentiment and usage of artificial intelligence tools. To analyse feedback given to the respondents about their perceptions and experience, the author categorized key themes from the comments as illustrated in Table 6. These findings are derived from primary data collected through a structured online survey, with the original response distributions illustrated in Appendix 2 Figure 22-25).

Table 6. Developer Perceptions and Experiences with Generative AI from primary data

Theme	Positive Comments	Negative Comments
Efficiency & Task Assistance	"Generative AI helps with code snippets and debugging, improving my workflow." "Speeds up development and allows me to focus on complex issues."	"Generative AI is not the best but helps developers improve."
Collaboration & Creativity	"It sparks new ideas and improves collaboration within the team."	-
Job Displacement Fears	-	"Sometimes, I feel that the next version of AI will replace me as an IT professional."
Security Concerns	-	"We can't use AI tools like GitHub Copilot because of information security concerns."
Hype vs Reality	"It's a game-changer for my development workflow."	"Generative AI is more hype than reality. The novelty will wear off, and disappointment will follow."
Usefulness of AI in Development	"I recommend utilizing AI tools in software development." "AI helps improve productivity and creativity."	"I don't find generative AI to be useful enough to replace traditional methods."

Developers show a mixed feeling towards generative artificial intelligence tools. While many developers report increasing productivity and job satisfaction, others express scepticism about the

reliability of AI tools, particularly in complex and critical areas. There is a strong sentiment of caution when it comes to sensitive information, with developers taking steps to sanitize code and data before using AI tools. In overall, the positive and negative comments balanced each other in primary data while secondary data shown in Table 7 shows the same trend. Below are tables showing the positive and negative comments both from primary and secondary data.

Table 7. Perception and experience of developers from secondary data

Perception/Experience	**Study source**	**Example/explanation**
Trust and Reliability	Stack Overflow (2024), Ulfsnes et al. (2024)	Developers remain sceptical about the trustworthiness of AI, particularly for complex tasks that require deep understanding.
Positive Feedback	GitHub (2023)	Developers report increased productivity, better code quality, and enhanced satisfaction with AI tools, especially for routine tasks.
Caution in AI Usage	Ulfsnes et al. (2024)	Developers exercise caution when using AI for sensitive data or proprietary code, preferring to sanitize their inputs before using AI tools.

4.5 Future of Generative AI in SDLC

Both primary and secondary data in

Table 8 and Table 9 show that Integration of AI tools into the SDLC will continue to grow, particularly in testing and maintenance. AI tools will become an essential part of daily software development task, leading to more automated development process. While AI may reduce the need for certain traditional roles e.g., junior developers, manual testers, it will also create new positions that focus on the management and oversight of AI tools. These roles will involve the quality and ethical standards of AI outputs. The following

Table 8 shows the key future themes for generative AI in the software development life cycle based on the respondents' comments showed in Appendix 2 Figure 24.

Table 8. Future themes from Primary data

Theme	Examples from Respondents' Comments
Widespread Integration	"AI will become essential for every developer." "More and more people will use AI to replace traditional searching." "AI tools will be integrated into development workflows, improving collaboration and efficiency."
AI and Workforce Transformation	"AI will encourage productivity in developers." "It's going to be an assistance that every developer needs." "Developer adaptation will be required as AI becomes more significant."
Human-AI Collaboration	"AI will allow developers to focus more on creativity and innovation." "It will improve collaboration within teams." "AI tools will help automate tasks, giving developers more time for complex problems."
Specialization of AI Tools	"AI will become more specialized, like GitHub Copilot, focusing on predicting code." "Expect more specialized versions of AI rather than generic tools like ChatGPT."
Security & Ethical Considerations	"Ethical issues must be addressed, especially regarding data security." "I hope to see on-premises implementations to enhance data security." "AI tools can be accessed by anyone, making personal information vulnerable."
Increased Automation	"AI will automate tasks like requirements gathering and testing." "More automated test suites and programming/development." "AI will enhance SDLC by automating routine operations."
Impact on Decision Making	"AI will provide real-time analytics for better decision-making." "AI will enable faster code generation and improve decision-making in the development process."

According to secondary data in Table 9 below, human oversight will remain crucial. Human judgment will still be required, especially in creative problem solving and ensuring the ethical application of AI.

Table 9. Future trends from secondary data

Future Trend	Study Source	Example/Explanation
Widespread Integration	GitHub/Wakefield Research (2023)	AI tools are expected to become more integrated into testing, documentation, and other SDLC phases. 80% of developers anticipate deeper integration.
AI and Workforce Transformation	McKinsey (2024), IJCRT (2023)	AI could reduce the need for certain roles but create new ones, such as AI trainers and AI overseers. Developers will shift to higher-level roles that focus on innovation.
Human-AI Collaboration	MITRE/NIST (2024)	The future of AI in SDLC will require a balance between automation and human creativity, with AI tools supporting, but not replacing, human insight and oversight.

5 Discussion

5.1 Ethical Foundation and Reliability

The author follows the ethical principles and practices issued by the Jyväskylä University of applied Science. This includes, respecting other people works by citing their publication in an appropriate manner (JAMK University of Applied Sciences, n.d).

Ensuring the reliability and validity of the study is a central focus, as these principle helps minimize bias and ensure the accuracy of the results. Reliability refers to how consistently a method measures what its intended to measure. Simply put, if the same methods are applied in the same conditions, they should give consistent results (McCombes, 2023).

Validity refers to the extent to which a method or measurement accurately reflects what it is supposed to measure. In other words, if research has high validity, its result truly represents the real-world characteristics, properties or variations it aims to assess (Middleton, 2025).

While this study is aimed for reliable data collection, there a few limitations that could affect the validity of the results. These were solved in different ways.

A primary concern is the bias in the data. Despite the study goal of obtaining a global perspective, majority of the response came from one region creating a regional bias. This would distort the findings, as the sample does not represent the broader, global audience. With the sample size being relatively small, this issue is further amplified, limiting how much the results can be generalized to other wider population.

To address this, the study used a concurrent triangulation design, meaning it combined multiple data collection methods at the same time to cross-check and validate the results. By doing this, the study aimed to balance out the weaknesses of one method with the strengths of another, making the findings more reliable overall. However, while this approach strengthens the study, the regional bias sample size still points to the need for future research.

In addition to the regional bias, the study faced limitation due to time constraints and limited access to diverse data sets due to the concept of AI in SDLC being relatively new, affecting the ability to gather a more representative sample. The time constraints restricted the scope of data collection, preventing a broader range of responses from different regions and professional backgrounds.

Similarly, the limited access to varied data sources meant that the study could not fully capture the impact of generative AI In the SDLC. As a result, the findings may not be fully representative of the global developer community. These limitations should be considered when interpreting the study conclusions, and future research should aim to overcome these barriers for more diverse an accurate result.

Given these limitations, future studies could focus on expanding the geographical reach and increasing the sample size to ensure that the results are more representative of the global audience. More diverse data would improve the external validity of the findings and make more applicable to a wider range of contexts. In future studies, a mixed method approach combining qualitative interviews with quantitative surveys would allow for deeper insights and stronger cross-validation, helping to identify nuances that numbers alone may not capture.

The author acknowledges the use of AI, such as ChatGPT and Microsoft Copilot, for grammar correction and brainstorming. However, all text, themes, and ideas in this work are entirely original and belong to the author, meaning that no part of the work was created or significantly improved using Artificial Intelligence.

5.2 Discussion of the main results in view of Concurrent Triangulation Design

The discussion aims to analyse the impact of generative AI on the software development life cycle by examining both quantitative data and insights from secondary data sources. By converging and contrasting these perspectives, we seek to highlight AI roles in various software development life cycle phases, its positive and negative impact, developer's perceptions and potential future trajectory. This discussion provides a comprehensive understanding of how AI is transforming Software Development Life Cycle while acknowledging its limitations and challenges.

5.2.1 Adoption rate of Generative AI in Software Development Life Cycle.

Both secondary and primary data show convergent insight. In the coding and implementation phase, quantitative data has highest weight average score of 1.92 which shows that AI has been implemented more in coding, with 40% saying it significantly helps. Secondary data shows that tools like GitHub Copilot and ChatGPT speed up coding and help with quality, letting developers focus on creative tasks, both data agree AI makes coding faster and more efficient.

In testing and Debugging phase, quantitative data shows that testing has a score of 1.48 which is moderate and in secondary data, AI helps in finding errors faster, run test and suggest fixes.

In quantitative data, maintenance has a score of 1.32, showing moderate adoption and in secondary data AI helps generate code comments and documentation which is part of the maintenance and deployment phase, saving time. Both data show that AI helps with maintenance and task like documentation but does not fully understand the context.

Both secondary and primary data show contrasting Insights. In requirement Gathering & Analysis phase Quantitative data shows the lowest score of 0.95, suggesting that AI has not been adopted in this phase. In Secondary sources, there is less emphasis on AI role in Requirement Gathering. While AI can be helpful, its contribution in this phase appears to be minimal, possibly due to the complex and subjective nature of requirements analysis, which requires a deep understanding of context and domain knowledge, and stakeholders needs that AI tools currently cannot fully replace.

In the Design phase qualitative data shows a weighted average of 1.68, showing that 44% of respondents report a moderate adoption while 24% see a significant impact which shows overall moderate implementation. In Secondary Data AI role in design is not mentioned much in other sources. This could indicate that AI impact on design is growing but still not as strong in coding or testing.

According to insights above the data could suggest that AI is changing SDLC, with the most implemented in coding and testing, making them faster and better. Secondly, humans are still needed, especially for complex tasks. AI is still developing in design and maintenance, but it’s not perfect yet. Requirement gathering is the least impacted by AI, as it requires human decisions.

5.2.2 Positive and Negative Impacts

Both quantitative and qualitative data source show that AI significantly improve overall productivity (weighted average score of 2.00 for efficiency and 2.08 for productivity). On the other hand, AI has a moderate impact on innovation (1.60) and decision making (1.56), helping with faster feedback and insights, but not fully transforming these areas yet. Both data sets agree that

AI improves collaborations by enabling quicker feedback, knowledge sharing and innovation driven teamwork.

By contract, the data from the survey reveals that data quality (3.00) and integration (3.00) present significant challenges in the use of generative AI. Secondary data sources also highlight similar concerns, such as AI struggling with handling complex data or generating code that meets high- quality standards. However, secondary sources place greater emphasis on security and ethical concerns, such as data privacy and bias in AI -generated content, which are not as prominent in the survey quantitative results.

Second, the negative impact analysis reveals concerns about job displacement, with the survey noting potential job reductions for junior developers and manual testers. However, secondary sources place greater focus on the broader ethical implications of AI adoption, emphasizing concerns about AI potential to replace human roles in routine tasks, this suggest that while the survey indicates some concerns, secondary data highlights these issues in more depth.

According to insights above, the data could suggest that AI improves efficiency and productivity in SDLC. However, there are still issues with fully integrated AI and trusting its results. Ethical concerns and data quality problems are important challenges that need to address for AI to be more effectively in development.

5.2.3 Perceptions and Experiences of Developers Regarding Generative AI

Both quantitative and qualitative data show that majority of developers are using or planning to use AI tools, especially for tasks like coding and testing, suggesting high acceptance of AI in the field. Both quantitative data and qualitative data show that developers use generative AI for task assistance which in end improves productivity and efficiency. Both data show that AI supports smoother teamwork.

Both data show some contrasting insights. In quantitative data, developers are expressing concern about job displacement, especially for junior developers and manual testers, fearing AI could replace these roles. In Secondary data however, the resources focus more on the broader ethical implications of AI, emphasizing how AI adoption could replace human roles in routine tasks across

the development process. These sources provide a deeper exploration of this concern, including long term effects on employment in the technology industry.

In survey data, developers are cautious about AI reliability, particularly for complex tasks. Many trusts AI for simpler tasks like code generation or documentation but hesitate when it comes to critical or nuanced coding tasks. Secondary, source mention a similar scepticism abouts AI ability to handle complex task. However, they stress that AI inability to deeply understand context and its tendency to generate imperfect code could cause significant trust issues, especially in production level environments. This deeper concern is not as prominent in the survey data.

Security concerns are more heavily discussed in secondary sources. Where experts highlight risks such as data privacy violations, AI bias and potential misuse of proprietary information. The survey data only touches on this lightly, but secondary sources provide a more thorough examination of these risks, reflecting broader industry debates.

A portion of developers AI is a “game-changer”, improving their work processes while other feel that AI is overhyped and won’t live up to expectations in the long term. Secondary source also highlights the divide but go deeper into concern about AI limitations in the real world. These sources argue that while AI tools promise efficiency gains, they may fall short when applied to more complex development scenarios, suggesting that the initial excitement might fade as developers encounter AI limitations.

According to the insights above, the data could suggest that developers largely recognize the benefits of AI tools in boosting efficiency. collaboration, and productivity, though scepticism remains about their reliability, especially for complex tasks. Trust, security and fears of job displacement are common concerns. Overall, the high adoption rates show an eagerness to embrace AI, but developers remain cautious, especially regarding sensitive data and tasks requiring deeper understanding.

5.2.4 Future of Artificial Intelligence in SDLC

Both the survey and secondary data agree that AI will become a big part of software development, especially for tasks like coding and testing. Developers believe AI will improve efficiency by

automating routine tasks and helping teams work better together. Both sources also agree that AI will allow developers to focus on more creative tasks and innovation, rather than just repetitive work. Human-AI collaboration will be important, where AI helps developers but doesn't replace them.

Both data contrast in few ways. The survey data shows concern about AI taking jobs, like junior developers and manual testers, but secondary data paints a more positive picture, saying that AI will create new roles for developers to manage and oversee AI tools. Both data sets agree that AI will automate many tasks, but the secondary data places more focus on security and ethical concerns, such as privacy and misuse of data, while the survey mainly talks about AI improving productivity.

According to the insights above, the data may suggest that AI will play a major role in the future of software development, helping with automation, collaboration, and creativity. While there are concerns about job loss and ethics, both sources agree that AI will make developers' work easier and let them focus on more important, creative tasks. However, human oversight will still be necessary to make sure everything works well and stays secure.

5.3 Reconciling Theoretical Frameworks and Data collected.

The Diffusion of Innovation theory explains how developers are adopting generative AI. Respondents were aware of its benefits and found it helpful for tasks like coding and debugging. Positive feedback encourages more people to try it. This aligns with the stages of adoption outlined in DOI theory. However, issues like security and ethics remain challenges.

The Task-Technology Fit (TTF) model focuses on how well generative AI fits with developers' tasks. It works well for boosting productivity and creativity but faces limits, such as security concerns, that reduce its use.

Together, these theories show that while generative AI is widely used (DOI), its future success depends on addressing issues to better meet user needs (TTF). Respondents believe AI will play a bigger role in software development, helping automate tasks and improve processes, if current challenges are solved.

5.4 Recommendation for Future Research

A future study could focus on Improving AI understanding of human context and its ability to interact with stakeholders effectively. This could involve natural language processing advancement, where AI systems are trained to better interpret complex requirement and collaborate with human teams to gather comprehensive information.

Developing AI tools that aid in requirement gathering These tools could help teams brainstorm ideas, collect user needs, and suggest smart ways to organize discussions and gather information more efficiently.

Testing the effectiveness of AI-driven tools during early SDLC phases in real-world environments, comparing the results to traditional methods of requirement gathering to measure whether AI's involvement can improve accuracy, reduce misunderstandings, and ultimately lead to better outcomes.

6 Conclusion

This study examined the impact of Generative Artificial Intelligence on the Software Development Life Cycle by employing a mixed methods approach through concurrent triangulation design. By integrating both primary data from survey responses and insights from secondary literature, the research aimed to provide a comprehensive view of AI's current and potential role across various SDLC phases.

The findings reveal that generative AI tools are most influential in the coding and implementation phase, where they significantly enhance development speed, productivity, and task efficiency. The testing and maintenance phases also moderately benefit from AI, particularly in error detection. However, the impact of AI is considerably limited in earlier phases such as requirement gathering and system design. These stages involve a high degree of human judgment, contextual understanding, and stakeholder interactions, elements that current AI tools struggle to replicate or support meaningfully.

Trends in adoption show increasing use of AI tools, particularly for routine and repetitive tasks. Developers are actively incorporating AI into their workflows, with a strong preference for tasks like code generation and testing. Despite this growing trend, adoption remains uneven across SDLC phases. While technical stages are seeing rapid integration of AI, early phase activities such as requirement phase and stakeholder collaboration still rely heavily on human expertise.

Looking toward the future, both primary and secondary data support the idea that generative AI will become more embedded in software development processes. Developers anticipate that AI will continue to automate mundane tasks and enhance collaboration, allowing teams to focus on creative problem-solving and innovation. However, concerns remain about job displacement, especially among junior roles, and the ethical implications of AI adoption. While AI may introduce efficiencies, it also necessitates the creation of new roles focused on oversight, tool management, and ethical compliance.

In terms of developer perceptions, the study indicates a general acceptance of AI as a productivity-enhancing tool, particularly for straightforward tasks. Nonetheless, developers remain cautious about relying on AI for complex or high-stakes activities. Trust, reliability, and contextual awareness are still significant limitations. Secondary sources reinforce this sentiment, emphasizing

the importance of human oversight, especially when AI-generated content may affect critical systems.

The study also identifies a balance between the benefits and challenges of AI adoption. On the one hand, AI contributes to increased productivity, reduced workload, and improved communication within teams. On the other, significant challenges include issues with data quality and integration, as well as ethical concerns surrounding security and data privacy. While these concerns were more deeply explored in secondary sources, the primary data also acknowledged these risks to a lesser extent.

Overall, the findings suggest that generative AI has a transformative role to play in software development, particularly in enhancing efficiency and enabling new forms of collaboration. However, to realize its full potential, further advancements are needed in enabling AI to better understand human context, interact more effectively with stakeholders, and support complex cognitive tasks. Future research should explore these dimensions, with particular focus on the early SDLC phases, where human-centered processes remain dominant and AI's role is still emerging.

References

Abreu, G. de. (2023, September 4). Preliminary literature review: A guide for effective research. *Preliminary Literature Review Blog*. https://preliminary-literature-review-blog

Alek. (n.d.). *Understanding quantitative research: A brief overview and process* [PDF]. The Department of English Education, Faculty of Educational Sciences, UIN Syarif Hidayatullah Jakarta. https://repository.uinjkt.ac.id/dspace/bitstream/123456789/68933/1/Understanding%20Quantitative%20Research-%20A%20Brief%20Overview%20and%20Process.pdf

Amazon Web Services. (n.d). *What is SDLC? - Software Development Lifecycle explained*. https://aws.amazon.com/what-is/sdlc/

Alenezi, M., & Akour, M. (2025). AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions. *Applied Sciences*, *15*(3), 1344. https://doi.org/10.3390/app15031344

Bannon, T., & Laplante, P. (2024). Generative AI in the software development lifecycle. *Computer, 57*(12), 27. https://doi.org/10.1109/MC.2024.3474789

Bartneck, C., Lütge, C., Wagner, A., & Welsh, S. (2021). *What is AI? In An introduction to ethics in robotics and AI* (pp. 5–16). Springer. https://doi.org/10.1007/978-3-030-51110-4_2

Black, R., Davenport, J., Olszewska, J., Rößler, J., Smith, A. L., Wright, J., & Smith, A. L. (Ed.) (2022). *Artificial Intelligence and Software Testing: Building Systems You Can Trust*. BCS Press. https://shop.bcs.org/store/221/detail/workgroup?id=3-221-9781780175768

Braun, V., & Clarke, V. (2012). Thematic analysis. In H. Cooper, P. M. Camic, D. L. Long, A. T. Panter, D. Rindskopf, & K. J. Sher (Eds.), *APA handbook of research methods in psychology, Vol. 2: Research designs: Quantitative, qualitative, neuropsychological, and biological* (pp. 57–71). American Psychological Association. https://doi.org/10.1037/13620-004

Chappell, D. (2008). *What is application lifecycle management?* Microsoft Corporation. https://download.microsoft.com/download/9/5/F/95FB4775-F769-477E-B187-6737DC09BC9D/What%20is%20ALM--Chappell.pdf

Dakhel, A. M., Nikanjam, A., Khomh, F., Desmarais, M. C., Washizaki, H., & Abrahamsson, P.; N. (2024). *An Overview on Large Language Models*. Cham: Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-55642-5_1

Daniel Russo. 2024. Navigating the Complexity of Generative AI Adoption in Software Engineering. *ACM Transaction on software Engineering and Methodology*, 33(5), Article 135. https://doi.org/10.1145/3652154

de Vicente Mohino, J., Bermejo Higuera, J., Bermejo Higuera, J. R., & Sicilia Montalvo, J. A. (2019). The Application of a New Secure Software Development Life Cycle (S-SDLC) with Agile Methodologies. *Electronics*, *8*(11), 1218. https://doi.org/10.3390/electronics8111218

del Águila, I. M., Palma, J., & Túnez, S. (2014). Milestones in Software Engineering and Knowledge Engineering History: A Comparative Review. *The Scientific World Journal,*692510. https://doi.org/10.1155/2014/692510

Delve, Ho, L., & Limpaecher, A. (2022, March 23). Qualitative, quantitative, and mixed methods. Delve. https://delvetool.com/blog/mixedmethods

Durmuş, M., Ustoglu, I., Tsarev, R., & Börcsök, J. (2018). Enhanced V-Model. *Informatica, 42*(4). https://doi.org/10.31449/inf.v42i4.2027

Foote, K.D. (2024, March 5). A brief history of generative AI. *Dataversity*.

Frost**, J.** (2025, January 29). *Weighted average: Formula & calculation examples.* Statistics by Jim. https://statisticsbyjim.com/basics/weighted-average/

GeeksforGeeks. (2024). *Agile SDLC: Software Development Life Cycle. GeeksforGeeks*. https://www.geeksforgeeks.org/agile-sdlc-software-development-life-cycle/

GitHub. (2025, March 9). *Survey reveals AI's impact on the developer experience*. GitHub Blog. https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/

Glasow, P. A. (2005). *Fundamentals of survey research methodology* (MP 05W0000077). MITRE Corporation. https://www.mitre.org/sites/default/files/pdf/05_0638.pdf

Gunasekare, U. L. T. P. (2015). Mixed research method as the third research paradigm: A literature review. *International Journal of Science and Research (IJSR)*, 2015. https://ssrn.com/abstract=2735996

Gurung, G., Shah, R., & Jaiswal, D. (2020). Software development life cycle models—a comparative study. International Journal of Scientific Research in Computer Science, Engineering and Information Technology, 6(4), 30–37. https://doi.org/10.32628/CSEIT206410

https://www.dataversity.net/a-brief-history-of-generative-ai/

JAMK University of Applied Sciences. (n.d.). *Requirements of ethical thinking*. JAMK University of Applied Sciences. https://oppimateriaalit.jamk.fi/en/ethics/introduction-to-professional-ethics/values-and-beliefs/requirements-of-ethical-thinking/

Jotform Editorial Team. (2024, October 30). *The pros and cons of Google Forms*. Jotform. https://www.jotform.com

Kaminski, J. (2011, Spring). Diffusion of Innovation Theory. *Canadian Journal of Nursing Informatics, 6*(2). Theory in Nursing Informatics Column. https://cjni.net/journal/?p=1444

Khan, S. M. A. (2023). *V-Model used in software development* (Technical Report No. SRE-010). National University of Computer and Emerging Sciences

Kumar, S. (2024, May 6). Understanding the application and impact of AI in software development life cycle. DZone. https://dzone.com/articles/application-and-impact-of-ai-in-the-sdlc?utm_source=chatgpt.com

Marikyan, D., & Papagiannidis, S. (2023). Task-Technology Fit: A review. In S. Papagiannidis (Ed.), *TheoryHub Book*. Available at https://open.ncl.ac.uk / ISBN: 9781739604400

McCombes, S. (2019). *Descriptive research: Definition, types, methods & examples*. Scribbr. Retrieved March 10, 2025, from https://www.scribbr.com/methodology/descriptive-research/

McCombes, S. (2023, June 22). *Descriptive research | Definition, types, methods & examples.* Scribbr. https://www.scribbr.com/methodology/descriptive-research/

McKinsey & Company. (2024). *Unleashing developer productivity with generative AI*. McKinsey. https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/unleashing-developer-productivity-with-generative-ai

Middleton, F. (2025, January 14). *Reliability vs. validity in research | Difference, types and examples.* Scribbr. https://www.scribbr.com/methodology/reliability-vs-validity/

Mishra, A. (2013). A comparative study of different software development life cycle models in different scenarios. International Journal of Advance Research in Computer Science and Management Studies, 1, 64–69. https://www.researchgate.net/publication/289526047_A_Comparative_Study_of_Different_Software_Development_Life_Cycle_Models_in_Different_Scenarios

Mohammad, H. (2023). Software development life cycle (SDLC) methodologies for information systems project management. *International Journal of Future Generation Communication and Networking, 5*(5). https://doi.org/10.36948/ijfmr.2023.v05i05.6223

Muthukrishnan, N., Maleki, F., Ovens, K., Reinhold, C., Forghani, B., & Forghani, R. (2020). Brief History of Artificial Intelligence. *Neuroimaging clinics of North America*, *30*(4), 393–399. https://doi.org/10.1016/j.nic.2020.07.004

Pothukuchi, A. S., Kota, L. V., & Mallikarjunaradhya, V. (2023). Impact of generative AI on the software development lifecycle (SDLC). International Journal of Creative Research Thoughts, 11(8), 5 pages. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4536700

Prakash, M. (2024). *Role of generative AI tools (GAITs) in software development life cycle (SDLC): Waterfall model*. [master's thesis, Massachusetts Institute of Technology]. MIT Libraries. https://dspace.mit.edu/bitstream/handle/1721.1/154009/mridulaprakash-mridula-sm-sdm-2024-thesis.pdf?sequence=1&isAllowed=y.

Prakash, M. (2024). *Role of Generative AI tools (GAITs) in Software Development Life Cycle (SDLC)-Waterfall Model* [Master's thesis, Massachusetts Institute of Technology]. Massachusetts Institute of Technology. https://hdl.handle.net/1721.1/154009

Red Hat. (2020, June 18). *What is application lifecycle management (ALM)?* Red Hat. https://www.redhat.com/en/topics/devops/what-is-application-lifecycle-management-alm

Ruparelia, N. B. (2010). Software development lifecycle models. *SIGSOFT Software Engineering Notes*, *35(3),* 8–13. https://doi.org/10.1145/1764810.1764814

Saravanos, A., & Curinga, M. X. (2023). Simulating the Software Development Lifecycle: The Waterfall Model. *Applied System Innovation*, *6*(6), 108. https://doi.org/10.3390/asi6060108

Saumure, K., & Given, L. (2008). Convenience sample. In The SAGE encyclopedia of qualitative research methods (Vol. 0, pp. -). SAGE Publications, Inc., https://doi.org/10.4135/9781412963909

Silva Martins, F., Carneiro da Cunha, J., & Serra, F. (2018). Secondary data in research – Uses and opportunities. *Revista Ibero-Americana de Estratégia*, *17*, 01-04. http://dx.doi.org/10.5585/ijsm.v17i4.2723

Smith, J., & Wei, R. (2024). Implications of artificial intelligence in the software development life cycle. *EasyChair*. https://easychair.org/publications/preprint/XVTt/open

Smith, J., & Wei, R. (2024). *Implications of artificial intelligence in the software development life cycle* (EasyChair Preprint No. 12604). EasyChair. https://easychair.org/publications/preprint/XVTt

SolDevelo. (2020, November 26). Is Agile always the best solution for software development projects? [Image]. https://soldevelo.com/blog/is-agile-always-the-best-solution-for-software-development-projects/

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Stack Overflow. (2024). *2024 Stack Overflow developer survey: AI* [Webpage]. Retrieved from https://survey.stackoverflow.co/2024/ai#efficacy-and-ethics

Stryker, C., & Kavlakoglu, E. (2024, August 9). What is AI? *IBM*. https://www.ibm.com/think/topics/artificial-intelligence

Svyatkovskiy, A., Deng, S. K., Fu, S., & Sundaresan, N. (2020). IntelliCode Compose: Code generation using transformer. *In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)* (pp. 1433–1443). Association for Computing Machinery. https://doi.org/10.1145/3368089.3417058

Taherdoost, H. (2021). Authors' biography. *International Journal of Academic Research in Management, 10*(1), 38. http://www.elvedit.com

TutorialsPoint. (2024). *V-Model*. TutorialPoint. https://www.tutorialspoint.com/sdlc/pdf/sdlc_v_model.pdf

Ulfsnes, R., Moe, N. B., Stray, V., & Skarpen, M. (2024). Transforming software development with generative AI: Empirical insights on collaboration and workflow. In A. Nguyen-Duc, P. Abrahamsson, & F. Khomh (Eds.), *Generative AI for effective software development* (pp. 159–177). Springer. https://doi.org/10.1007/978-3-031-55642-5_10

United Nations. (n. d). The 17 goals. United Nations. https://sdgs.un.org/goals

Van Pham**.** (2024). *Applying AI in software development lifecycle process.* KMS Solutions. Retrieved March 6, 2025, from https://kms-solutions.asia/blogs/applying-ai-in-software-development-lifecycle-process?hs_amp=true

Wilkinson, L. (2023, September 8). AI adoption in software development is slow to take root. *GitLab*. https://www.gitlab.com/blog/ai-adoption-in-software-development

Appendices

Appendix 1 The Impact of Generative AI on Software Development Life Cycle survey

The survey requests the participant's email if needed to send the results of the thesis.

*Required

Background and Demographics

1. What region do you work in? *

Mark only one oval.

- Africa,
- Asia,
- Europe,
- North America,
- Oceania,
- South America,
- others

2. How many years have you been involved in software development? *

Mark only one oval.

- Less than 1 year,
- 1-4 years,
- 5-9 years,
- 10-14 years,
- 15+ years,
- others

3. Does your organisation use Generative AI tools in the software development process? *

Mark only one oval.

- Yes,
- No,

4. What is your current role in software development?

Mark only one oval.

- Developer (Full-stack, Back-end, Front-end, Mobile, etc.),
- Data Engineer,
- AI/ML Specialist,
- DevOps Specialist,
- Product Manager,
- others

5. What SDLC model do you use in your organisation or team? *

Mark only one oval.

- Waterfall,
- V-model (Verification and Validation),
- Incremental Model,

- Agile Model,
- DevOPs,
- others

Awareness and Adoption of Generative AI

6. How familiar are you with generative AI technologies (e.g., machine learning, natural language processing? *

Mark only one oval.

- Not familiar at all
- Slightly familiar
- Moderately familiar
- Very familiar
- Extremely familiar

7. Which phases of the SDLC process is generative AI tools used (Select all that apply)? *

- Requirement identification and gathering
- Design and Architecture
- Development and programming
- Testing
- Deployment and maintenance

Impact on SDLC Phases: Requirement Gathering and Analysis, Design and Architecture, Coding and Implementation, Testing and QA, Deployment and Maintenance

Requirement Gathering and Analysis

8. what extent has generative AI influenced the requirement gathering and analysis phase of your projects? *

Mark only one oval.

- No impact
- Slight impact
- Moderate impact
- Significant impact
- Transformative impact

Design and Architecture

9. How has generative AI affected the design and architecture phase of your software development? Mark only one oval. *

- No impact

- Slight impact
- Moderate impact
- Significant impact
- Transformative impact

Coding and Implementation

10. What level of automation or assistance has generative AI provided in the coding and implementation phase? *

Mark only one oval.

- No impact
- Slight impact
- Moderate impact
- Significant impact
- Transformative impact

11. How has generative AI impacted your testing and QA processes? *

Mark only one oval.

- No impact
- Slight impact
- Moderate impact
- Significant impact
- Transformative impact

Deployment and Maintenance

12. In what ways has generative AI improved the deployment and maintenance of your software? *

Mark only one oval.

- No impact
- Slight impact
- Moderate impact
- Significant impact
- Transformative impact

Efficiency and Productivity

13. Has generative AI increased efficiency gains in your development workflow? *
Mark only one oval.
- No contribution
- Minor contribution
- Moderate contribution
- Significant contribution
- Major contribution

14. To what extent has generative AI improved your overall productivity? *
Mark only one oval.
- No improvement
- Slight improvement
- Moderate improvement
- Significant improvement
- Major improvement

Decision-Making and Innovation

15. Has generative AI enhanced decision making in your projects? *
Mark only one oval.
- Not at all
- Slightly
- Moderately
- Significantly
- Extremely

16. In your opinion, how much has generative AI fostered innovation in your software development processes? *
Mark only one oval.
- Not at all
- Slightly
- Moderately
- Significantly
- Extremely

Challenges and Ethical Considerations

17. What are the main challenges you have faced when integrating generative AI into your SDLC? *

    Select all that apply
    - Data quality issues
    - Lack of skilled personnel
    - Integration with existing tools
    - Ethical concerns
    - Others

18. How concerned are you about the ethical implication of using generative AI in software development? *

    Mark only one oval.
    - Not concerned
    - slightly concerned
    - Moderately concerned
    - Very concerned
    - Extremely concerned
19. How would you describe your overall experience with generative AI tools? *

    Mark only one oval.
    - Very Positive
    - Positive
    - Neutral
    - Negative
    - Very Negative
20. How do you foresee the role of AI evolving in the SDLC over the next five years? *

    Long answer text

21. Would you recommend the use of generative AI tools to other developers? *

    Mark only one oval.
    - 1
    - 2
    - 3
    - 4
    - 5
22. Why would you recommend AI tools or why not? *

    Long text answer…

23. Any additional comments or experiences you'd like to share regarding generative AI in software development? *

    Long text answer

24. Would you like to receive the survey results? if yes please leave your email below.

- Yes
- No

Short answer text.

Appendix 2 Survey results

Awareness and Adoption of Generative AI

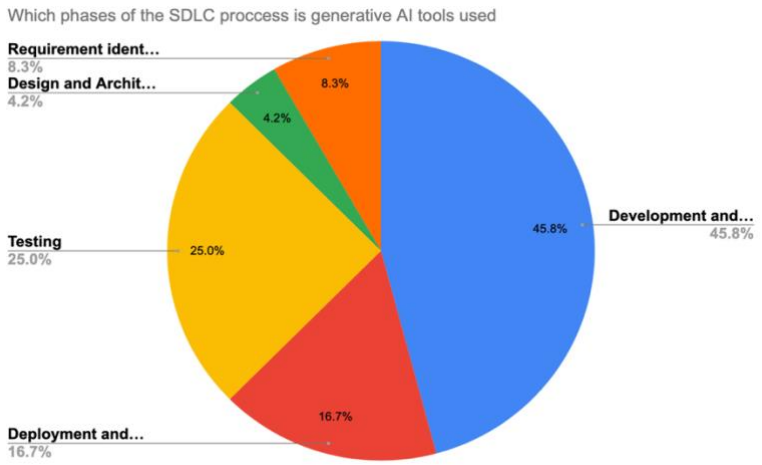

Figure 11. Phases of the SDLC that implement AI

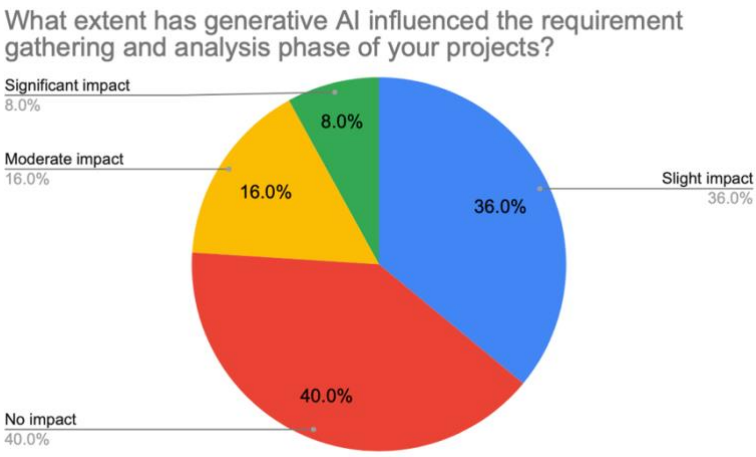

Figure 12. Impact of GenAI in Requirement and Gathering

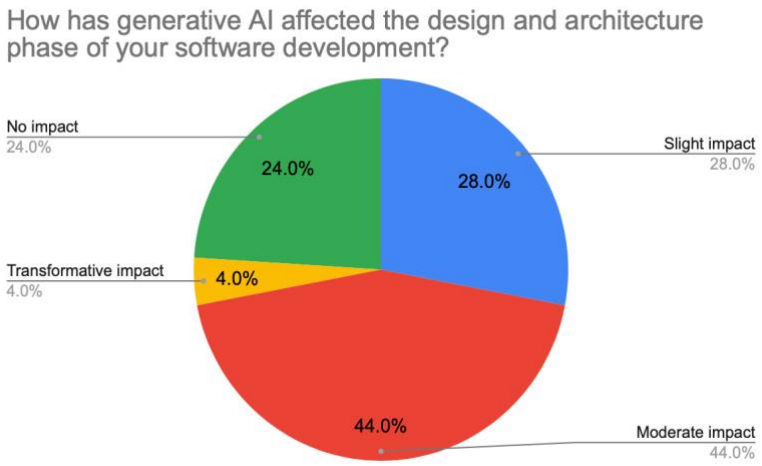

Figure 13. Impact in design and architecture Phase

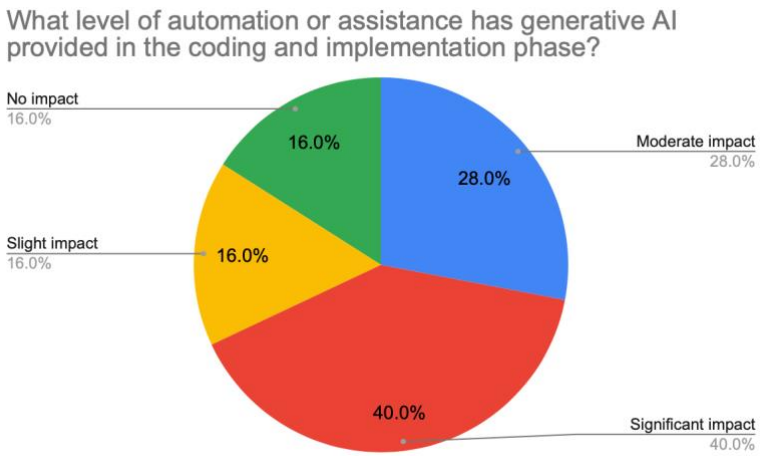

Figure 14. Impact in coding and implementation phase

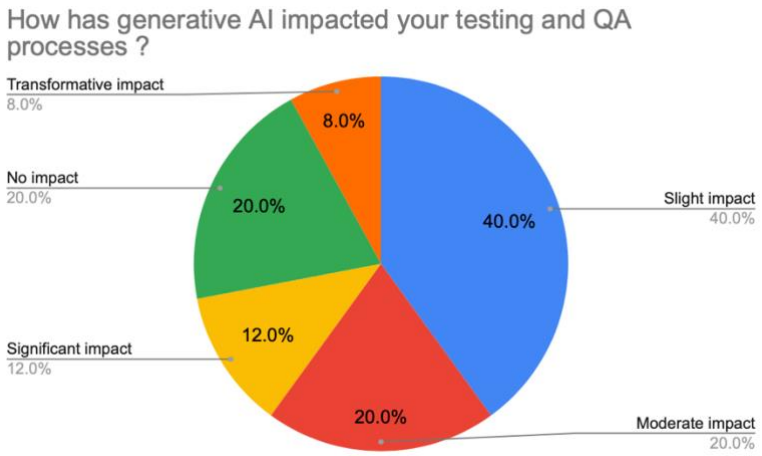

Figure 15. Impact in testing and QA Phase

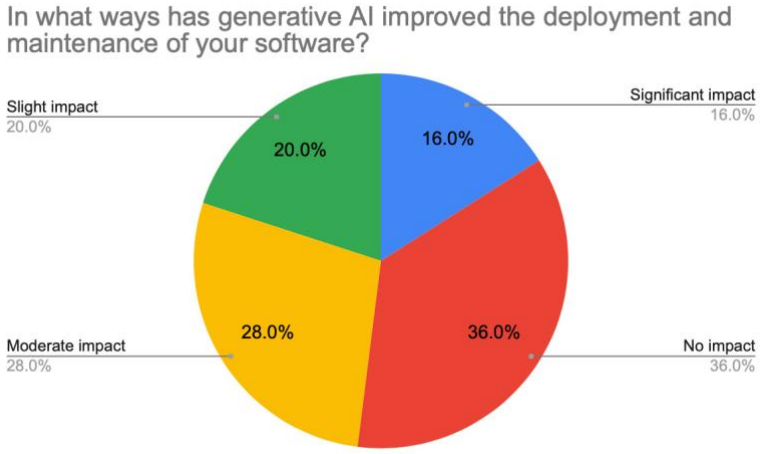

Figure 16. Deployment and maintenance Phase

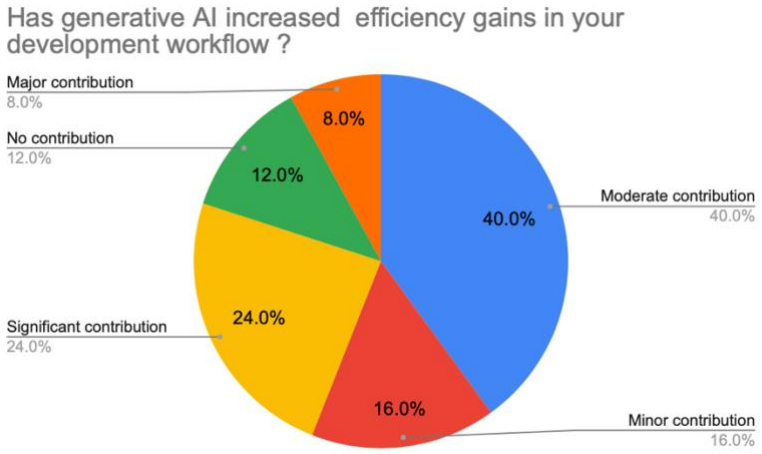

Figure 17. Efficiency in development workflow results

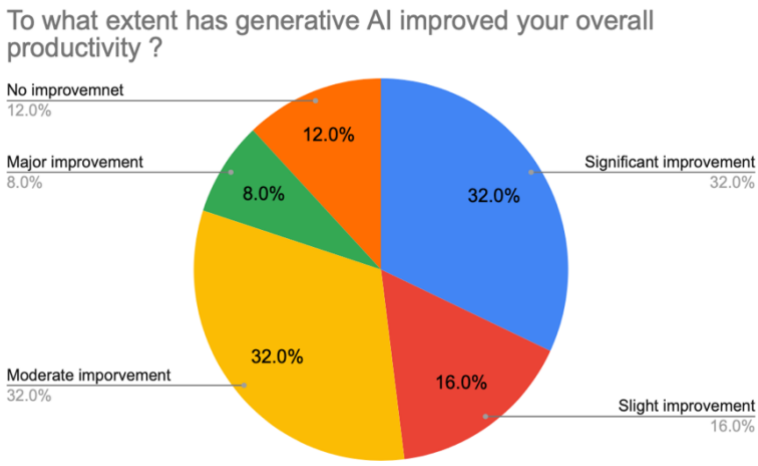

Figure 18. Productivity improvement results

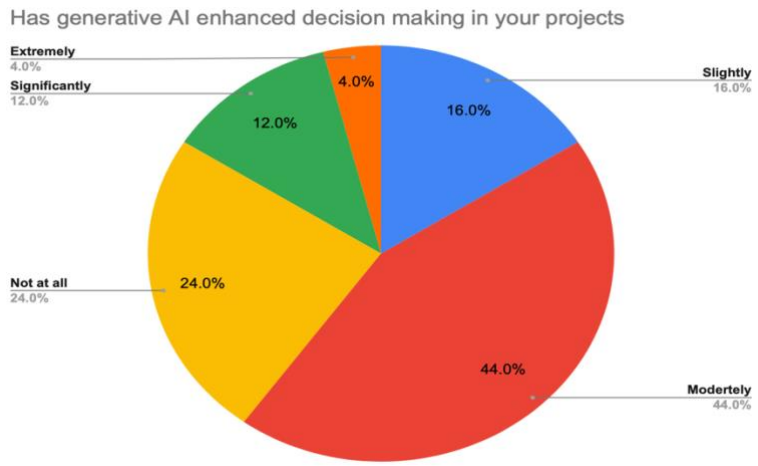

Figure 19. Decision making results

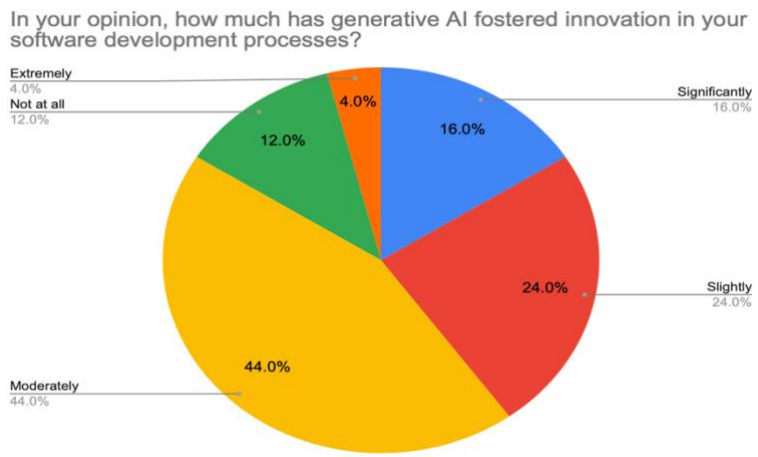

Figure 20. Impact of AI In innovation results

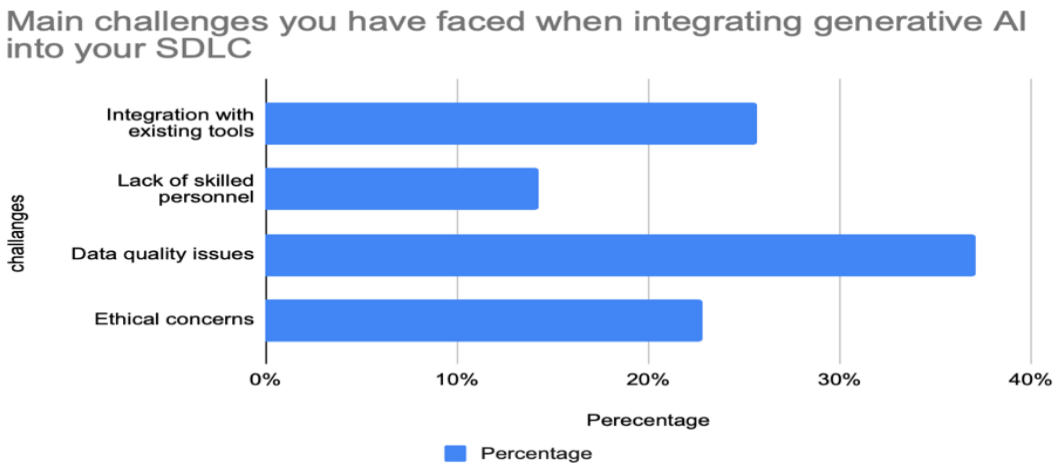

Figure 21. Challenges in integrating Gen AI into SDLC results

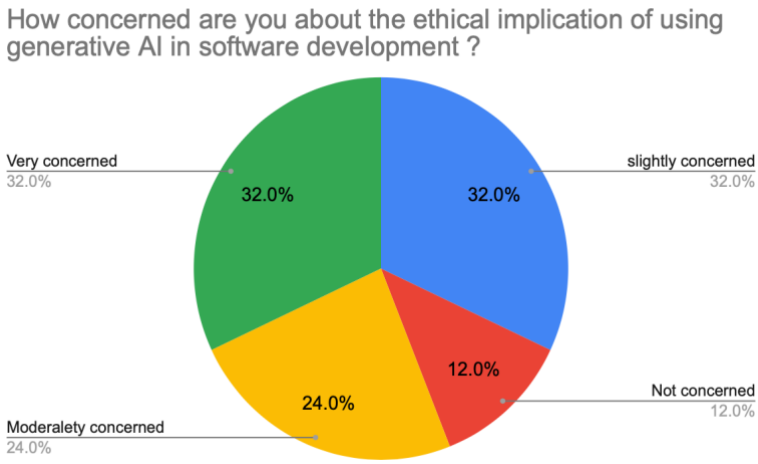

Figure 22. Ethical concerns results

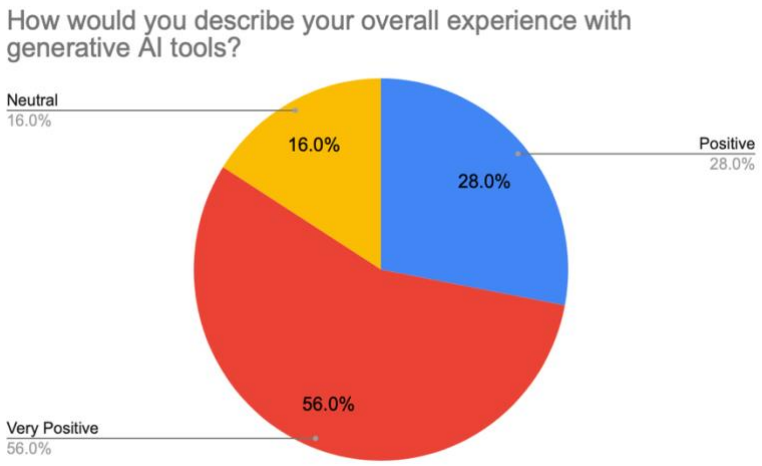

Figure 23. Overall experience using generative AI results

"no :)"

"Sometimes, you have a feeling that next version will replace you as an IT professional, and this bring more challenges"

"I recommend to utilize AI tool in software development"

"Generative ai is not the best but it helps developers to become better with its help."

Figure 24. Perceptions of respondents about the future of AI in SDLC

"I've found generative AI to be a game-changer in my development workflow. It quickly helps with code snippets and debugging, allowing me to focus on more complex problems. The ability to experiment with different coding approaches has also sparked new ideas and improved collaboration within my team. Overall, it's an exciting tool that enhances both efficiency and creativity."
-
"A lot of hype is around Generative AI because of the novelty value - the intelligent chatbot that is ChatGPT shows potential, but translating generative AI into concrete business cases is a lot more difficult. The hype will very likely die down during the coming years, as some of the novelty wears off, and the general disappointment will start to show in how generative AI was not the general AI that the general public thought it was. Nevertheless, some useful technologies and techniques will emerge out of all of this, and will benefit us in the long run. "

"One major widespread concern/limitation with current generative AI scene is the information security side. As I mentioned previously, our team would significantly benefit from use of generative AI in our work (specifically GitHub Copilot or similar), but we are not allowed to utilize the technology because of information security concerns. Until the functionality of the AI can be completely restrained within a private environment, generative AI is a tool that is ruled out for many companies that deal with sensitive information."

"No"

Figure 25. Comments and experiences shared regarding generative AI in software development by the respondents